Babeș-Bolyai University of Cluj-Napoca
Faculty of Mathematics and Computer Science
Computer Science Specialization

A Privacy-Preserving, Federated Anomaly Detection Framework for Distributed Microservice Architectures

Bachelor's Thesis
Submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science

Scientific Coordinator
Prof. Dr. Sergiu Adrian Dărăbanț

Student
Moghioroș Eric

May 5, 2026

Universitatea Babeș-Bolyai din Cluj-Napoca
Facultatea de Matematică și Informatică
Specializarea Informatică

Un Cadru Federat de Detecție a Anomaliilor cu Păstrarea Confidențialității pentru Arhitecturi Distribuite de Microservicii

Lucrare de Licență

Coordonator Științific
Prof. Dr. Sergiu Adrian Dărăbanț

Absolvent
Moghioroș Eric

May 5, 2026

Abstract

Modern distributed systems, composed of microservices and heterogeneous components, are increasingly complex and prone to performance anomalies that can compromise reliability and service quality. This thesis presents a comprehensive anomaly detection framework designed for large-scale distributed systems, integrating real-time edge detection, federated learning for collaborative model training, causal graph-based root cause analysis, and adaptive, performance-aware thresholding. By combining localized anomaly detection at the edge with global model aggregation via the FedAvg algorithm, the framework enhances detection accuracy while preserving data privacy through Differential Privacy. The incorporation of causal graphs enables precise identification of root causes, facilitating rapid remediation and reducing alert latency. Adaptive thresholding powered by Q-learning dynamically adjusts detection sensitivity according to system performance metrics, improving responsiveness under varying workloads. Experimental evaluation demonstrates that the proposed approach achieves high detection fidelity, low false-positive rates, and scalable performance across distributed architectures, offering a robust solution for proactive system reliability management.

Contents

1 Introduction **10**

1.1 Motivation 10

1.2 Problem Statement 10

1.3 Research Objectives 11

1.4 Research Contributions 11

2 Background and Literature Review **13**

2.1 Distributed Systems 13

2.1.1 Definition and Characteristics 13

2.1.2 The CAP Theorem 14

2.1.3 Common Failure Modes 14

2.2 Microservices Architecture 15

2.2.1 Definition and Comparison with Monolithic Architecture 15

2.2.2 Containerization and Orchestration 15

2.3 Observability in Distributed Systems 15

2.3.1 The Three Pillars: Metrics, Logs, Traces 15

2.3.2 Anomaly Detection Approaches 16

2.4 Related Work 16

2.4.1 Anomaly Detection in Microservices 16

2.4.2 Federated Learning for System Monitoring 17

2.4.3 Root Cause Analysis Systems 17

2.4.4 Adaptive Thresholding and Reinforcement Learning 17

3 Edge Computing and Metrics Collection **18**

3.1 Edge Computing Paradigm 18

3.1.1 Definition and Characteristics . . . 18
3.1.2 Edge vs. Cloud Computing . . . 18
3.1.3 Edge Computing in Distributed Systems . . . 19
3.2 Metrics Collection . . . 19
3.2.1 What Metrics to Collect . . . 19
3.2.2 How to Collect Metrics . . . 20
3.2.3 Metric Preprocessing . . . 21
3.3 Local Anomaly Detection at the Edge . . . 21
3.3.1 On-Device Inference . . . 21
3.3.2 Statistical Baseline Methods . . . 22
3.3.3 Machine Learning Methods at the Edge . . . 22

4 LSTM-Autoencoder Model Architecture 24
4.1 Recurrent Neural Networks and Their Limitations . . . 24
4.2 Long Short-Term Memory (LSTM) . . . 25
4.2.1 LSTM Architecture . . . 25
4.2.2 LSTM for Anomaly Detection . . . 26
4.3 Autoencoders . . . 26
4.3.1 Autoencoder Architecture . . . 26
4.3.2 Why Autoencoders for Anomaly Detection . . . 26
4.4 LSTM-Autoencoder Architecture . . . 27
4.4.1 Combined Architecture . . . 27
4.4.2 Encoder LSTM . . . 27
4.4.3 Latent Space . . . 27
4.4.4 Decoder LSTM . . . 28
4.4.5 Linear Output Layer . . . 28
4.5 Bidirectional LSTM . . . 28
4.5.1 Architecture . . . 28
4.5.2 Advantages and Limitations for Edge Deployment . . . 29

5 Federated Learning with Differential Privacy 30
5.1 Introduction to Federated Learning . . . 30
5.1.1 Motivation . . . 30

5.1.2 Federated Learning Paradigm 31
5.2 Federated Learning Architecture 31
5.2.1 Participants: Edge Nodes and Central Coordinator 31
5.2.2 Aggregation Strategies 32
5.2.3 Training and Evaluation in a Federated Setting 32
5.2.4 Model Deployment to Edge Nodes 33
5.3 Differential Privacy 34
5.3.1 Motivation for Differential Privacy 34
5.3.2 How Differential Privacy Works 34
5.3.3 Differential Privacy in Federated Learning 35
5.4 Fault Tolerance in Federated Learning 36
5.4.1 Adaptive Node Profiles 36
5.4.2 Handling Node Dropout and Late Joiners 37

6 Communication: Serialization and Compression 38
6.1 Message Serialization 38
6.2 Message Transport 39
6.3 Message Compression 40
6.3.1 Top-K Gradient Sparsification 40
6.3.2 Byte-Level Compression 40
6.3.3 Adaptive Compression Selection 41

7 Root Cause Analysis Engine 42
7.1 Motivation 42
7.2 Distributed Tracing 42
7.2.1 Trace Collection 42
7.2.2 Trace Structure: Spans and Context 43
7.3 Dependency Graph and Causal Graph 43
7.3.1 Service Dependency Graph Construction 43
7.3.2 Causal Graph 44
7.3.3 Root Cause Indicators in Causal Graphs 45
7.4 Root Cause Scoring 46
7.4.1 PageRank-Based Root Cause Scoring 46

7.4.2 Ranking and Presenting Results . . . 46
7.5 Implementation . . . 47
7.5.1 Framework Components . . . 47
7.5.2 Integration with Anomaly Detection . . . 48

8 Adaptive Thresholding **49**
8.1 Motivation . . . 49
8.2 Fixed vs Adaptive Thresholding . . . 50
8.3 Reinforcement Learning for Threshold Adaptation . . . 50
8.3.1 Reinforcement Learning Formulation . . . 50
8.3.2 Q-Learning . . . 51
8.3.3 Q-Learning Implementations and Variants . . . 51
8.3.4 Disadvantages and Limitations . . . 52
8.4 Implementation . . . 52
8.4.1 Architecture . . . 52
8.4.2 Integration with the Detection Pipeline . . . 53
8.4.3 Example: Dynamic CPU Utilization Alert Threshold . . . 54

9 Reliability and Fault Tolerance **55**
9.1 Reliability in Distributed Systems . . . 55
9.2 CAP Theorem Implications . . . 55
9.3 Single Points of Failure . . . 56
9.3.1 Identification . . . 56
9.3.2 Mitigation Strategies . . . 56
9.4 Redundancy . . . 57
9.4.1 Data Redundancy . . . 57
9.5 Fault Tolerance Mechanisms . . . 58
9.5.1 Crash Detection . . . 58
9.5.2 Edge Node Profiles . . . 58

10 System Design and Implementation **59**
10.1 End-to-End System Architecture . . . 59
10.1.1 High-Level Architecture Overview . . . 59
10.1.2 How Components Fit Together . . . 60

10.2 Data Flow . . . 61
10.2.1 Metric Ingestion Pipeline . . . 61
10.2.2 Anomaly Detection Pipeline . . . 61
10.2.3 Federated Learning Pipeline . . . 62
10.2.4 Root Cause Analysis Pipeline . . . 62
10.3 Component Design . . . 63
10.3.1 Edge Node Agent . . . 63
10.3.2 Central Coordinator . . . 63
10.3.3 Trace Collector . . . 64
10.3.4 Alerting and Dashboarding . . . 64
10.4 System Orchestration . . . 65
10.4.1 Kubernetes Deployment . . . 65
10.4.2 Configuration Management . . . 65
10.4.3 Managing Data Flow and Scheduling . . . 66
10.5 Implementation Details . . . 66
10.5.1 Technology Stack . . . 66
10.5.2 Key Implementation Decisions . . . 67

11 Experimental Evaluation **68**
11.1 Experimental Setup . . . 68
11.1.1 Hardware and Software Environment . . . 68
11.1.2 Datasets . . . 69
11.1.3 Baselines . . . 69
11.2 Anomaly Detection Performance . . . 70
11.2.1 Metrics . . . 70
11.2.2 Detection Accuracy Results . . . 70
11.2.3 Impact of Differential Privacy . . . 72
11.3 Federated Learning Communication Overhead . . . 73
11.3.1 Payload Size and Compression . . . 73
11.3.2 Transmission Latency . . . 74
11.3.3 Adaptive Compression Effectiveness . . . 75
11.4 Root Cause Analysis Accuracy . . . 76
11.4.1 RCA Evaluation Methodology . . . 76

11.4.2 Root Cause Ranking Results 76
11.5 Adaptive Thresholding Performance 78
11.5.1 Alert Fatigue Reduction 78
11.5.2 Q-Learning Convergence 79
11.6 Scalability 80
11.6.1 Scaling Edge Nodes 80
11.6.2 Large-Scale Distributed Environments 80

12 Conclusion and Future Work **82**
12.1 Summary of Contributions 82
12.2 Conclusion 83
12.3 Limitations 84
12.4 Future Work 85
12.4.1 Transfer Learning 85
12.4.2 Hierarchical Federated Learning 85
12.4.3 Enhanced Privacy Mechanisms 85
12.4.4 Improved Root Cause Analysis 86
12.4.5 Deep Q-Network for Adaptive Thresholding 86
12.4.6 Multi-Modal Anomaly Detection 86

References **88**

List of Figures

10.1 High-level architecture of the distributed anomaly detection framework. Edge agents communicate with the central coordinator via gRPC. Arrows indicate primary data flows; dashed lines indicate broadcast. 59

11.1 LSTM-AE hyperparameter sweep comparison across six configurations. 71

11.2 Cold-start convergence: MSE loss (left) and F1-score (right) over 100 training rounds. The model achieves effective anomaly separation after approximately 50–60 rounds. . 72

11.3 Privacy–accuracy tradeoff: AUC-ROC vs. training round (left) and loss convergence (right) under different DP noise levels. 72

11.4 Payload size heatmap across serialization and compression combinations. 74

11.5 Stacked FL round latency breakdown showing local training dominance. 75

11.6 RCA accuracy: Framework vs. Random baseline on Train-Ticket and Alibaba-scale topologies. 77

11.7 Adaptive threshold strategies over time: threshold value (top) and cumulative alerts (bottom) across a simulated 24-hour window. 79

11.8 FedAvg aggregation scalability: time (log-log, left) and memory delta (right) vs. number of edge nodes. 80

List of Tables

2.1 Comparison of Microservices and Monolithic Architectures 15

3.1 Comparison of edge and cloud computing paradigms 19

6.1 Comparison of serialization formats for federated model payloads 39

6.2 Conceptual comparison of LZ4 and Zstandard based on reported literature 41

10.1 Technology stack used in the framework implementation 67

11.1 Hardware and software environment used for all benchmarks 68

11.2 Overall anomaly detection performance on the SMD dataset. The federated model with optimized hyperparameters (Config C) significantly outperforms both baselines. 70

11.3 LSTM-AE hyperparameter sweep results on SMD. Config C (h=128, L=2) achieves the highest F1 of 0.839. 71

11.4 Impact of differential privacy on detection performance after 200 training rounds. AUC-ROC degrades monotonically with increasing noise, showing a clear privacy–accuracy trade-off. 72

11.5 Gradient clipping norm sensitivity at $\sigma = 0.01$. AUC-ROC is insensitive to the clipping norm across the tested range, suggesting that the Gaussian noise dominates clipping effects at this noise level. 73

11.6 Payload size and compression performance for the best LSTM-AE model (h=128, L=2, 475K parameters). TorchSave (binary) produces payloads 5.4× smaller than text-based formats. Zstd L1 achieves the best ratio on JSON/Protobuf (61% reduction). 74

11.7 Per-round FL latency breakdown (5 clients, h=128 model). Local training dominates total round time (> 97%). Compression and aggregation are negligible. 75

11.8 Adaptive compression algorithm selection under varying CPU loads. The controller switches from Zstd to LZ4 at 80% CPU, trading 8.7% compression for a 5.8× speedup. 76

11.9 RCA accuracy comparison across two topologies. The framework outperforms the random baseline on Top-1 (+7 pp) and MRR (+0.052) on the Train-Ticket topology. . 77
11.10 Cumulative alert counts and false positive rates for three thresholding strategies across a 24-hour simulation. 78
11.11 Q-learning agent state evolution. The agent accumulates positive reward from true negatives, but requires more training time to lower its threshold and detect anomalies. 79
11.12 FedAvg aggregation scalability from 5 to 1 000 nodes. Aggregation time scales approximately linearly ($\approx$ 1.0 ms per node). 80
11.13 Full FL round latency breakdown at 20 and 100 nodes. Local training dominates at all scales; aggregation remains sub-30 ms even at 100 nodes. 81

Chapter 1

Introduction

1.1 Motivation

The contemporary digital ecosystem is characterized by an accelerating proliferation of distributed systems that have become the de facto standard for modern software delivery [23]. Primarily instantiated through microservice-based deployments [71, 25], this paradigm decomposes monolithic applications into independently deployable, loosely coupled services. While this delivers substantial benefits in development velocity, scalability, and fault isolation, it simultaneously introduces unprecedented complexity in system observability and anomaly detection [52]. Traditional centralized monitoring paradigms, adequate for monolithic architectures, are rendered fundamentally insufficient in this context [59].

1.2 Problem Statement

Detecting anomalies across highly decentralized deployments introduces significant technical challenges. The sheer volume of telemetry generated by thousands of concurrent service instances creates a data ingestion bottleneck [91], while the ephemeral nature of cloud-native environments renders static monitoring configurations obsolete [59]. Maintaining strict service level objectives (SLOs) in this volatile environment necessitates a paradigm shift: from reactive, centralized aggregation toward proactive, privacy-preserving, and intelligent distributed monitoring [59].

1.3 Research Objectives

The primary objective of this research is to design and evaluate a decentralized architecture for anomaly detection in distributed systems that guarantees operational visibility without compromising data privacy. This breaks down into five concrete sub-objectives:

1. **On-Device Model Architecture**: Design a lightweight LSTM-Autoencoder (LSTM-AE) [65] for localized anomaly detection at the edge.

2. **Privacy-Preserving Collaborative Training**: Integrate federated learning [67] with differential privacy guarantees [26, 1] for robust global model training without exposing local telemetry.

3. **Intelligent Root Cause Identification**: Construct a root cause analysis engine leveraging causal-dependency graphs to contextualize metrics and identify anomalous service interactions.

4. **Contextual Anomaly Thresholding**: Develop a Q-learning-driven [100] adaptive thresholding mechanism that optimizes anomaly sensitivity on a per-service basis.

5. **Optimized Infrastructure Communication**: Implement and benchmark model weight serialization pipelines contrasting Protocol Buffers with Zstandard and LZ4 compression [21, 20].

1.4 Research Contributions

This thesis delivers the following novel contributions to distributed systems observability and AIOps [59]:

- **Decentralized Anomaly Framework**: A federated LSTM-Autoencoder framework tailored for heterogeneous distributed environments.

- **Secure Aggregation Pipeline**: A differential-privacy-aware FedAvg [67] aggregation pipeline over operational telemetry.

- **Topological Root Cause Scoring**: A causal-dependency graph approach with a PageRank-inspired heuristic [76] for scalable fault attribution.

- **Reinforcement Learning for Thresholds**: Q-learning applied to autonomously manage adaptive thresholding, minimizing false positives and negatives per service.

- **Communication Cost Empirics**: Empirical evaluation of LZ4 and Zstd compression schemes in bandwidth-constrained federated deployments.

Chapter 2

Background and Literature Review

2.1 Distributed Systems

2.1.1 Definition and Characteristics

Distributed systems are collections of independent computers that appear to users as a single coherent system [95]. Their autonomous components communicate via message passing and may span diverse hardware, virtual machines, and geographic locations. The foundational characteristics of such architectures are:

- **Scalability**: The capacity to dynamically adapt to an expanding workload by adding distributed nodes (horizontal scaling) or enhancing existing node resources (vertical scaling) [38, 54].
- **Fault Tolerance**: The ability to sustain operational correctness despite component, node, or network failures, realized through structural redundancy and state replication [54, 95].
- **Transparency**: Abstraction of the distributed topology from applications and users, encompassing location transparency (hiding resource placement) and access transparency (uniform access to local and remote assets) [95, 22].
- **Concurrency**: Coordinated simultaneous access to shared resources by multiple independent processes, enforced through locking mechanisms or distributed consensus protocols [22, 54].
- **Heterogeneity**: Seamless integration across diverse hardware architectures, operating environments, and network infrastructures through standardized communication protocols [22, 95].

2.1.2 The CAP Theorem

The CAP Theorem, conjectured by Brewer [14] and formally proven by Gilbert and Lynch [36], states that any distributed data store can simultaneously guarantee at most two of the following three properties:

- **Consistency (C)**: Every read reflects the most recent write, making the distributed topology appear as a single logical entity to the client.
- **Availability (A)**: Every non-failing node responds to requests in a reasonable timeframe, without guaranteeing that the response reflects the most recent mutation.
- **Partition Tolerance (P)**: The system sustains continuous operation despite the loss or arbitrary delay of messages between network nodes.

Because network partitions are unavoidable in practice, architects must choose between CP systems (prioritizing consistency over availability) and AP systems (prioritizing availability while accepting transient data divergence) [36, 54].

2.1.3 Common Failure Modes

Distributed failures manifest across several fault domains [23]:

- **Network Failures**: Partitions, packet loss, latency spikes, and DNS resolution failures that disrupt inter-service communication.
- **Node Failures**: Hardware degradation, abrupt crash failures, and crash-recovery loops where a node continuously restarts, potentially losing volatile state.
- **Software and Service Failures**: Application bugs, deadlocks, misconfigurations, and resource exhaustion (CPU, memory, or thread pool starvation).
- **Data and Consistency Issues**: Stale reads due to replication lag, split-brain scenarios during partitions, and Byzantine failures where nodes exhibit arbitrary or malicious behavior [54, 84].

2.2 Microservices Architecture

2.2.1 Definition and Comparison with Monolithic Architecture

Microservices decompose applications into suites of small, autonomous services, each encapsulating a singular business capability with its own codebase and persistence layer [71]. Services communicate exclusively via well-defined network APIs, enabling independent deployment and scaling [25]. Compared to monolithic architectures—which co-locate all logic into a single deployable artifact—microservices offer superior fault isolation, independent horizontal scalability, and polyglot technology diversity, at the cost of significantly increased operational complexity in monitoring, data consistency, and network overhead.

Aspect	**Microservices**	**Monolithic**
Structure	Small, independent services	Single, tightly integrated codebase
Deployment	Independent per service	Deployed as one unit
Scalability	Per-service	Entire application uniformly
Fault Tolerance	High (failure isolation)	Low (single point of failure)
Complexity	Operationally complex	Structurally simple

Table 2.1: Comparison of Microservices and Monolithic Architectures

2.2.2 Containerization and Orchestration

Microservice deployment relies on containerization (primarily Docker [68]), which isolates applications and their dependencies into lightweight, immutable environments. Orchestration platforms such as Kubernetes [16] automate the lifecycle of containerized workloads, providing automated rollouts, service discovery, internal load balancing, and self-healing mechanisms.

2.3 Observability in Distributed Systems

2.3.1 The Three Pillars: Metrics, Logs, Traces

Observability is defined by a system's capacity to be understood through its external outputs [52]. It is realized through three interconnected telemetry pillars:

- **Metrics**: Aggregated numerical health indicators (e.g., CPU utilization, latency percentiles), collected by tools such as Prometheus [82] and visualized in Grafana [39].

- **Logs**: Immutable, timestamped records of discrete computational events, aggregated via the ELK Stack (Elasticsearch, Logstash, Kibana) [28].
- **Traces**: Reconstructions of the request lifecycle across service boundaries, enabling engineers to pinpoint bottlenecks [91], implemented by tools such as Jaeger [49] and Zipkin [75].

True observability links a high-level symptom (e.g., a metric anomaly indicating increased latency) to its source pathway (e.g., a trace revealing a downstream timeout), and finally to the granular cause (e.g., a log exposing a database connection failure).

2.3.2 Anomaly Detection Approaches

Automated anomaly detection spans a wide methodological spectrum:

1. **Rule-Based Thresholds**: Static operational limits that are computationally trivial but prone to high false-positive rates under organic traffic spikes.
2. **Statistical Methods**: Moving averages, rolling standard deviations, and z-scores that establish dynamic baselines, though they struggle with complex multivariate anomalies.
3. **Predictive Analytics**: Leveraging historical seasonality to forecast expected metric behavior.
4. **Machine Learning**: Unsupervised models (e.g., clustering) and deep learning (e.g., Autoencoders, LSTMs [65]) capable of detecting minute, multidimensional deviations across vast data streams [63].

2.4 Related Work

2.4.1 Anomaly Detection in Microservices

Contemporary research emphasizes unsupervised and semi-supervised machine learning for microservice environments, given the rarity of labeled fault data [59]. Statistical bounds have largely been superseded by sequential neural models, particularly LSTM-Autoencoders, which exhibit superior competence in modeling the temporal dependencies of service metrics.

2.4.2 Federated Learning for System Monitoring

Centralizing telemetry from geographically dispersed edge nodes incurs prohibitive bandwidth costs and introduces data privacy concerns. Federated Learning (FL) [67] addresses this by enabling edge nodes to train models locally and share only gradient updates with a central aggregator. Integrating Differential Privacy [26] into this pipeline [1] ensures that aggregated weights cannot be reverse-engineered to expose sensitive local telemetry.

2.4.3 Root Cause Analysis Systems

Prominent RCA systems leverage topological service graphs derived from distributed traces [91, 105]. Systems such as MicroRank construct causal dependency graphs and employ random-walk algorithms akin to PageRank [76] to quantitatively score and rank probable fault origins amid cascading failure alerts [57].

2.4.4 Adaptive Thresholding and Reinforcement Learning

Static detection thresholds degrade rapidly in volatile cloud environments. Reinforcement learning—specifically Q-learning [100]—models the thresholding problem as a Markov Decision Process, enabling agents to autonomously adapt sensitivity profiles in real-time by learning from historical alert outcomes [94, 62].

Chapter 3

Edge Computing and Metrics Collection

3.1 Edge Computing Paradigm

3.1.1 Definition and Characteristics

Edge computing brings computation and data storage closer to the sources of data generation, rather than relying exclusively on centralized cloud infrastructure [89, 86, 98, 66]. In the context of distributed and microservice-based systems, this means processing telemetry, logs, and service-level signals near the host or service instance where they originate. The defining characteristics of edge computing are:

- **Low Latency**: Processing data near the point of origin reduces the delay associated with transmitting information to remote cloud infrastructure before analysis or action is taken.
- **Bandwidth Efficiency**: Local preprocessing, filtering, and inference reduce the volume of raw data that must be transmitted over wide-area network (WAN) links.
- **Data Locality**: Keeping data near its source supports privacy preservation, operational control, and regulatory compliance.
- **Resilience and Autonomy**: Edge systems can continue to perform useful computation even under degraded or intermittent connectivity to centralized services [86].

3.1.2 Edge vs. Cloud Computing

Edge and cloud computing are generally regarded as complementary paradigms [89, 24]. Cloud infrastructure provides elastic compute capacity and centralized orchestration, while the edge emphasizes locality, reduced latency, and decentralized processing.

Dimension	Edge Computing	Cloud Computing
Processing Location	Near the data source	Remote centralized infrastructure
Latency	Lower for local decisions	Higher due to network transit
Bandwidth Usage	Reduced via local preprocessing	Higher when raw data is centralized
Data Privacy	Greater locality and control	Data aggregated centrally
Compute Capacity	Often resource-constrained	Highly elastic and scalable
Fault Scope	Failures often remain localized	Central failures affect larger portions

Table 3.1: Comparison of edge and cloud computing paradigms

In practice, real-time inference runs at the edge while long-term storage, global coordination, and large-scale model training are handled centrally. Edge computing is also a natural fit for federated learning, where model adaptation occurs across nodes without centralizing the underlying raw data [67, 51].

3.1.3 Edge Computing in Distributed Systems

Microservice architectures generate large volumes of fine-grained telemetry across many independently deployed components [16, 70, 25], making them natural candidates for edge-oriented monitoring. Edge-based analysis can be realized through several architectural patterns:

- host-level monitoring agents,
- service-local collectors,
- sidecar processes or containers,
- embedded telemetry modules.

The common principle across these patterns is that at least part of the collection, transformation, or inference pipeline executes near the monitored component, enabling local anomaly detection, streaming analysis, adaptive control, and privacy-preserving monitoring.

3.2 Metrics Collection

3.2.1 What Metrics to Collect

The effectiveness of anomaly detection depends strongly on the observability signals available to the detection model. In distributed systems, collected metrics typically fall into four broad categories:

- **Resource Utilization**: CPU utilization, memory consumption, disk activity, filesystem pressure, and process-level resource usage, reflecting infrastructure and runtime stress.
- **Network Performance**: Throughput, latency indicators, connection counts, packet-level characteristics, and retransmission behavior—particularly important in communication-heavy microservice systems.
- **Application Performance**: Request rate, error rate, response time, queueing delay, and service-level indicators (SLIs) reflecting the externally visible health and responsiveness of a service.
- **Internal Service Metrics**: Thread counts, garbage collection behavior, cache performance, queue depths, and connection pool utilization, exposing internal contention not visible at the infrastructure layer.

Anomaly detection benefits from combining infrastructure-level and application-level signals, since anomalous behavior may manifest as isolated metric deviations or as changes in the relationships between multiple correlated variables [91, 10, 64].

3.2.2 How to Collect Metrics

Metrics collection follows an instrumentation and acquisition pipeline where signals are exposed, sampled, transported, and stored [82, 97]. Common collection approaches include:

1. **Application-Level Instrumentation**: Services expose internal counters, gauges, histograms, or summaries via language-specific instrumentation libraries.
2. **Infrastructure and Kernel-Level Monitoring**: System metrics are collected using host-level exporters or low-level tracing mechanisms such as eBPF, which enables efficient kernel and syscall level observability [42].
3. **Pull-Based Collection**: Monitoring systems periodically scrape metric endpoints exposed by services or agents.
4. **Push-Based Collection**: Monitored components actively emit metric updates to downstream collectors or aggregation systems.

In anomaly detection pipelines, collected samples are organized into temporally ordered windows before being consumed by statistical or machine learning models, since temporal context is frequently necessary to distinguish normal variability from abnormal behavior [65, 47].

3.2.3 Metric Preprocessing

Raw operational metrics are often noisy, heterogeneous, and expressed on different numerical scales. Preprocessing is therefore required before the data can be used effectively in detection models. Common steps include:

1. **Normalization or Standardization**: Features are rescaled to comparable numerical ranges, which is critical for neural network models sensitive to scale differences across input dimensions.
2. **Temporal Window Construction**: Sequential observations are segmented into fixed-length windows to form model inputs suitable for time-series analysis.
3. **Missing Data Handling**: Incomplete time series arising from scrape failures or service restarts are addressed through interpolation, imputation, masking, or exclusion of incomplete samples.
4. **Noise Reduction and Smoothing**: Short-term volatility may be attenuated through smoothing or aggregation to emphasize persistent behavioral patterns.
5. **Feature Transformation**: Derived features such as rates of change, rolling statistics, or inter-metric ratios may be computed to expose temporal or relational structure not evident in raw signals.

Care must be taken to ensure that preprocessing does not inadvertently remove the very patterns intended for anomaly identification.

3.3 Local Anomaly Detection at the Edge

3.3.1 On-Device Inference

On-device inference executes anomaly detection models near the telemetry source rather than within a fully centralized pipeline. This approach offers several advantages:

- reduced response time for local anomaly identification,
- lower network utilization due to reduced raw-data transmission,
- improved privacy through local processing,
- increased operational autonomy under intermittent connectivity.

These properties are particularly relevant where centralized streaming analysis is constrained by latency, bandwidth, reliability, or privacy considerations [89, 86]. Edge inference is also compatible with federated learning, where model training occurs across nodes without centralizing the underlying raw data [67, 51].

3.3.2 Statistical Baseline Methods

Before considering more expressive machine learning models, anomaly detection is often approached using classical statistical methods, which are computationally lightweight and serve as useful baselines [7, 18]:

- **Moving Average**: Tracks local central tendency over a rolling window to identify deviations from recent behavior.
- **Rolling Standard Deviation**: Measures short-term variability and supports dynamic thresholding based on local dispersion.
- **Z-Score**: Quantifies how far an observation deviates from a reference mean relative to the estimated standard deviation.
- **Exponentially Weighted Moving Average (EWMA)**: Assigns greater weight to recent observations, commonly used in process monitoring and change detection [7].

While simple and interpretable, these methods are limited in their ability to model nonlinear dependencies, long-range temporal context, or multivariate interactions among correlated metrics.

3.3.3 Machine Learning Methods at the Edge

Machine learning methods extend statistical approaches by learning richer representations of normal and abnormal behavior. Commonly studied model families include:

- **Tree-Based Methods**: Isolation Forest isolates anomalies through recursive partitioning of the feature space [61].
- **Kernel-Based Methods**: One-Class SVM estimates a decision boundary enclosing normal observations in a kernel-induced feature space [87].

- **Neural Network Methods**: Deep learning models—feedforward autoencoders, recurrent networks, and sequence autoencoders—learn nonlinear and temporal patterns from multivariate telemetry [17, 77].

In sequential telemetry domains, temporal models are often advantageous because many anomalies are defined not by isolated values, but by unusual progressions or deviations from learned sequence dynamics.

Chapter 4

LSTM-Autoencoder Model Architecture

4.1 Recurrent Neural Networks and Their Limitations

Recurrent Neural Networks (RNNs) are a foundational class of neural architectures designed to process sequential data [85, 37]. Unlike feedforward networks, RNNs introduce recurrent connections that allow information to persist across time steps. At each step t, the network combines the current input x_t with the previous hidden state h_{t-1} to compute:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \tag{4.1}$$

where W_{xh} and W_{hh} are learnable weight matrices, b_h is a bias vector, and σ is a nonlinear activation (typically tanh).

Despite their expressiveness, vanilla RNNs suffer from well-known training instabilities when learning long-range dependencies [9, 79]. During Backpropagation Through Time (BPTT), repeated multiplication of weight matrices across many steps causes gradients to either vanish exponentially or grow without bound. The vanishing gradient problem is particularly harmful, as the error signal decays before it can influence earlier states. Gradient clipping [79] mitigates exploding gradients, but the vanishing gradient problem fundamentally motivates the use of gated architectures such as Long Short-Term Memory networks.

4.2 Long Short-Term Memory (LSTM)

4.2.1 LSTM Architecture

The LSTM network, introduced by Hochreiter and Schmidhuber [46, 40], addresses the vanishing gradient problem through a gating mechanism that regulates information flow within the recurrent cell. An LSTM cell maintains a cell state C_t (long-term memory) and a hidden state h_t (working memory), governed by three multiplicative gates:

- **Forget Gate** (f_t): Determines what to discard from the previous cell state:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \tag{4.2}$$

- **Input Gate** (i_t): Controls what new information is written, using a candidate cell state $\tilde{C}_t$:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \tag{4.3}$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \tag{4.4}$$

- **Output Gate** (o_t): Regulates which parts of the cell state are exposed as the hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \tag{4.5}$$

The cell and hidden states are then updated as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \tag{4.6}$$

$$h_t = o_t \odot \tanh(C_t) \tag{4.7}$$

where σ is the sigmoid function and $\odot$ denotes element-wise multiplication. The additive cell-state update provides a gradient highway that preserves information across long time horizons [34], making LSTMs significantly more stable than vanilla RNNs for learning long-range dependencies.

4.2.2 LSTM for Anomaly Detection

In time-series anomaly detection, LSTM-based models are trained on data representative of normal system behavior [65, 47, 29]. The model learns temporal regularities and multivariate relationships present in nominal operational patterns. During inference, sequences that deviate from the learned representation of normality typically produce higher reconstruction or prediction errors, enabling unsupervised fault detection without requiring labeled anomaly examples.

4.3 Autoencoders

4.3.1 Autoencoder Architecture

An autoencoder is an unsupervised neural network that learns to encode its input into a lower-dimensional latent representation and subsequently reconstruct the original input [45, 37]. The architecture consists of two complementary sub-networks:

- **Encoder** f_θ: Maps the input $x \in \mathbb{R}^d$ to a latent vector $z \in \mathbb{R}^k$, where $k < d$.
- **Decoder** g_ϕ: Reconstructs the input from the latent representation, producing $\hat{x} = g_\phi(z) \approx x$.

The model is trained end-to-end to minimize reconstruction error, typically using Mean Squared Error (MSE):

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2 \tag{4.8}$$

The bottleneck imposed by the latent dimension k forces the model to learn a compressed representation of the input distribution rather than merely replicating the raw input.

4.3.2 Why Autoencoders for Anomaly Detection

Autoencoders are widely used for anomaly detection because they can learn a representation of normal data without requiring explicit anomaly labels [17, 77]. Their suitability stems from several properties:

- **Unsupervised Learning**: They can be trained using only data representative of normal behavior, which is advantageous where anomalous examples are rare or difficult to label.
- **Multivariate Dependency Modeling**: They learn cross-feature relationships and compressed representations of high-dimensional data.

- **Reconstruction Error as Anomaly Score**: Since the model is optimized to reconstruct normal patterns, anomalous inputs that fall outside the learned distribution yield elevated reconstruction errors.

Compared to Principal Component Analysis (PCA)—a linear analog of the autoencoder—neural autoencoders capture nonlinear feature interactions, making them more expressive for complex telemetry signals [50, 17]. The autoencoder paradigm also extends naturally to sequential variants (e.g., LSTM-AE), whereas PCA offers no inherent temporal modeling.

4.4 LSTM-Autoencoder Architecture

4.4.1 Combined Architecture

The LSTM-Autoencoder (LSTM-AE) integrates the temporal modeling capacity of LSTMs with the unsupervised representation learning of autoencoders [65, 106], yielding a model well-suited to multivariate time-series anomaly detection. The processing pipeline is:

$$X \xrightarrow{\text{Encoder LSTM}} z \xrightarrow{\text{Repeat}} z^{(1..T)} \xrightarrow{\text{Decoder LSTM}} H \xrightarrow{\text{Linear}} \hat{X} \tag{4.9}$$

where $X \in \mathbb{R}^{T \times d}$ is the input sequence of T time steps with d features, $z \in \mathbb{R}^k$ is the compressed latent vector, and $\hat{X} \in \mathbb{R}^{T \times d}$ is the reconstructed sequence.

4.4.2 Encoder LSTM

The encoder LSTM processes the input sequence $X = (x_1, x_2, \ldots, x_T)$ sequentially and produces a latent representation from its final hidden state:

$$z = h_T \in \mathbb{R}^k \tag{4.10}$$

This representation is intended to capture the most salient temporal and multivariate characteristics of the input window.

4.4.3 Latent Space

The latent dimensionality k is a key hyperparameter governing the degree of compression. A latent space that is too large may reconstruct even anomalous inputs faithfully, reducing anomaly separability.

One that is too small may fail to preserve normal structure, leading to uniformly poor reconstruction. Its optimal value depends on data complexity and the required anomaly sensitivity.

4.4.4 Decoder LSTM

The decoder LSTM reconstructs the temporal sequence from z by repeating the latent vector across the time dimension:

$$z^{(1..T)} = (z, z, \ldots, z) \in \mathbb{R}^{T \times k} \tag{4.11}$$

This repeated sequence is processed by the decoder LSTM, producing a sequence of hidden states $H = (h_1^{\text{dec}}, \ldots, h_T^{\text{dec}})$ corresponding to the reconstructed temporal structure.

4.4.5 Linear Output Layer

A shared linear transformation projects each decoder hidden state back into the original feature space:

$$\hat{x}_t = W_{\text{out}} h_t^{\text{dec}} + b_{\text{out}} \tag{4.12}$$

yielding the final reconstruction $\hat{X} = (\hat{x}_1, \ldots, \hat{x}_T)$. The model is trained end-to-end by minimizing the MSE between X and $\hat{X}$.

The computational footprint of the LSTM-AE is governed by input dimensionality, hidden size, number of recurrent layers, and sequence length. In latency-sensitive or resource-constrained deployments, compact architectures that balance representational power with feasible inference cost are preferred.

4.5 Bidirectional LSTM

4.5.1 Architecture

A Bidirectional LSTM (BiLSTM) extends the standard LSTM by processing the input sequence in both forward ($t = 1 \rightarrow T$) and backward ($t = T \rightarrow 1$) directions using two separate recurrent layers [41, 88]. The hidden states from both passes are concatenated at each time step:

$$h_t^{\text{bi}} = [\overrightarrow{h_t} \| \overleftarrow{h_t}] \in \mathbb{R}^{2k} \tag{4.13}$$

This dual-direction processing yields richer temporal representations by incorporating both past and future context for each position in the sequence.

4.5.2 Advantages and Limitations for Edge Deployment

Bidirectional LSTMs offer greater representational capacity, but introduce practical trade-offs relevant to operational deployments:

- **Increased Computational Cost**: Bidirectional processing requires more parameters and more computation than a unidirectional architecture.
- **Higher Memory Usage**: Maintaining forward and backward hidden states increases runtime memory requirements.
- **Full Sequence Requirement**: Backward processing requires the complete input sequence to be available before inference can be performed, which may limit suitability for low-latency streaming scenarios.

The choice between unidirectional and bidirectional architectures therefore depends on the trade-off between representational richness and deployment constraints such as latency, memory, and online processing requirements.

Chapter 5

Federated Learning with Differential Privacy

5.1 Introduction to Federated Learning

5.1.1 Motivation

Centralized training in large-scale distributed systems presents several limitations:

- **Data Volume**: Aggregating raw telemetry or operational data from thousands of distributed nodes may exceed available network bandwidth and storage capacity, creating bottlenecks and increasing latency in model training [58].
- **Privacy Constraints**: In many industrial and regulatory contexts, sensitive telemetry or Personally Identifiable Information (PII) must not leave the local environment. Regulations such as the General Data Protection Regulation (GDPR) [78] impose legal obligations to limit cross-boundary data transfer [51].
- **Network Bandwidth Limitations**: Edge nodes often operate over constrained or intermittent networks. Continuously streaming raw datasets is infeasible, leading to delays or dropped data [12].
- **Security Risks**: Transmitting raw data increases the attack surface for adversaries who may intercept or tamper with sensitive information [101].

Federated Learning (FL) addresses these challenges by enabling nodes to train locally while sharing only model updates or gradients, significantly reducing communication overhead and improving data privacy [67].

5.1.2 Federated Learning Paradigm

Federated Learning is a distributed machine learning paradigm in which a global model is collaboratively trained across multiple clients, each holding private local datasets [67]. Let there be K clients, each with dataset $\mathcal{D}_k$. The global objective function is:

$$\min_{\theta} F(\theta) = \sum_{k=1}^{K} \frac{|\mathcal{D}_k|}{\sum_{j=1}^{K} |\mathcal{D}_j|} F_k(\theta), \tag{5.1}$$

where $F_k(\theta)$ denotes the local loss at client k. Each federated learning round comprises:

1. **Local Training**: Clients update model parameters θ using their local private data.
2. **Gradient or Weight Upload**: Clients transmit parameter updates $\Delta\theta_k$ or gradients $\nabla F_k(\theta)$ to the central server.
3. **Aggregation**: The server computes a global update by aggregating received updates.
4. **Broadcast**: The updated global model θ_{t+1} is distributed back to all clients for the next round.

This iterative process continues until convergence criteria, such as loss stabilization or a predefined number of rounds are met [58].

5.2 Federated Learning Architecture

5.2.1 Participants: Edge Nodes and Central Coordinator

The federated learning architecture comprises two participant roles.

Edge Nodes (Clients). Each edge node monitors a single microservice instance and maintains a local copy of the LSTM-based autoencoder (LSTM-AE) described in Chapter 4. During each FL round, the node collects a window of local telemetry—CPU utilization, memory consumption, request latency, and error counts—and trains the local model for a small number of epochs. The resulting parameter update $\Delta\theta_k$ is compressed using the adaptive serialization and compression pipeline described in Chapter 6 before being transmitted to the coordinator via gRPC. Between rounds, the local model continues to perform real-time inference independently, ensuring that anomaly detection is not interrupted by network partitions or coordinator downtime.

Central Coordinator (Server). The coordinator maintains the authoritative global model θ_t and orchestrates each FL round. It (i) registers edge nodes and tracks their participation state, (ii) receives

and buffers compressed model updates, (iii) verifies updates against a poisoning detection pipeline (Section 5.4), (iv) aggregates valid updates using FedAvg, and (v) broadcasts the updated global model back to all registered nodes. The coordinator persists its state—including the current round number, the global model checkpoint, and the client registry—to a durable database, allowing it to resume after a crash without losing progress (see Chapter 9).

5.2.2 Aggregation Strategies

This framework adopts the *Federated Averaging* (FedAvg) algorithm for global model aggregation, which is widely adopted due to its simplicity and communication efficiency [67].

FedAvg computes a weighted average of client updates according to their local dataset size. Formally, if $\theta_t^{(k)}$ is the updated model from client k after local training, the server computes:

$$\theta_{t+1} = \sum_{k=1}^{K} \frac{n_k}{\sum_{j=1}^{K} n_j} \theta_t^{(k)}, \tag{5.2}$$

where $n_k = |\mathcal{D}_k|$. This weighting ensures that clients with more data contribute proportionally to the global update. Under the assumptions of smooth loss functions, bounded gradients, and Independent and Identically Distributed (IID) or mildly non-IID data, FedAvg converges efficiently to a stationary point [58, 67].

FedAvg's properties make it particularly suitable for resource-constrained edge environments:

- **Low Communication Overhead**: Only model weights or gradients are transmitted, avoiding raw data movement.
- **Computational Simplicity**: Clients perform standard local Stochastic Gradient Descent (SGD) updates without additional complex computations.
- **Robustness to Moderate Heterogeneity**: Works effectively when client data distributions vary slightly [58].

5.2.3 Training and Evaluation in a Federated Setting

Each edge node performs local training using the following procedure.

1. **Initialization.** At the start of a round, the node loads the latest global model parameters θ_t received from the coordinator. The current parameters are saved as θ_t^{init} so that the model update $\Delta\theta_k = \theta_k - \theta_t^{\text{init}}$ can be computed after training.

2. **Local Training.** The node trains for $E = 5$ local epochs on its private dataset using the Adam optimiser with a learning rate of 10^{-3} and a batch size of 64. The training objective is reconstruction loss (MSE) between the input window and the LSTM-AE output.

3. **Gradient Compression.** After training, the parameter delta $\Delta\theta_k$ undergoes optional Top-K sparsification (Section 6.3) with a default compression ratio of 10%, retaining only the largest-magnitude updates and accumulating the remainder in a local error-feedback buffer.

4. **Differential Privacy.** If DP is enabled, gradients are clipped to a norm of $C = 1.0$ and Gaussian noise calibrated to the privacy budget ε is added before transmission (Section 5.3).

5. **Transmission.** The compressed, optionally noised update is serialized and adaptively compressed at the byte level before being transmitted to the coordinator over a gRPC channel.

Communication frequency. By default, nodes synchronize with the coordinator once per round. The coordinator waits up to a configurable timeout (default 120 s) for at least $K_{\min}$ client updates before triggering aggregation. Nodes that miss a round simply re-join in the next one.

Evaluation. After each aggregation, the global model can be evaluated on a held-out validation partition. In the benchmark experiments (Chapter 11), the global model is evaluated after every round, yielding F1, Precision, and Recall on a separate test set with injected anomalies.

5.2.4 Model Deployment to Edge Nodes

After each successful aggregation round, the updated global model is deployed back to all registered edge nodes.

Version management. The coordinator maintains a monotonically increasing round counter t. Each global model checkpoint is tagged with this round number and persisted to durable storage. A configurable retention policy keeps the last N checkpoints, enabling rollback if a newly aggregated model exhibits degraded detection quality.

Rollout strategy. Deployment follows a pull-based model: edge nodes request the latest global model from the coordinator via the `GetGlobalModel` gRPC endpoint. The coordinator responds with the serialized and compressed model state along with the round number. This approach avoids broadcast storms and allows nodes to update at their own pace.

Offline nodes. If a node was unreachable during a round, it is not penalized. When the node reconnects, it re-registers with the coordinator and downloads the latest global model. Its `last_seen`

timestamp is updated, and it is enrolled in subsequent rounds automatically. Staleness tolerance is configurable; updates from nodes whose local round number lags behind the global round by more than a configured tolerance (default 2 rounds) are discarded during aggregation.

Hot-swapping. The local model is swapped atomically: the node continues inference with the previous model weights until the new state dictionary has been fully loaded into memory. The swap operation involves a single call to `load_state_dict()`, after which the model resumes evaluation mode. Because torch operations are synchronous and the swap is in-memory, no inference windows are dropped during the transition.

5.3 Differential Privacy

5.3.1 Motivation for Differential Privacy

Even in federated learning, model updates can leak information about individual training samples:

- **Membership Inference**: An adversary may determine whether a specific sample was included in training by observing the changes in model parameters [90].
- **Model Inversion**: Sensitive attributes can be partially reconstructed from model weights or gradients [32].

Differential privacy (DP) provides a formal framework to bound the influence of any single sample on the released updates. Formally, a randomized mechanism $\mathcal{M}$ satisfies (ε, δ)-DP if for any pair of neighboring datasets D, D' differing in a single sample, and for all measurable subsets S of outputs:

$$\Pr[\mathcal{M}(D) \in S] \leq e^{\varepsilon} \Pr[\mathcal{M}(D') \in S] + \delta. \tag{5.3}$$

This ensures that an observer cannot confidently infer the presence or absence of any individual record from the mechanism's output [27, 1].

5.3.2 How Differential Privacy Works

DP mechanisms applied in gradient-based learning typically involve the following steps:

1. **Per-Sample Gradient Clipping**: For each sample x_i in a mini-batch of size B, the gradient $g_i = \nabla_\theta \mathcal{L}(\theta, x_i)$ is clipped to a threshold C:

$$\bar{g}_i = g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right), \tag{5.4}$$

limiting the influence of any single training example on the aggregate update.

2. **Noise Addition**: The aggregated gradient over the batch is perturbed with Gaussian noise:

$$\tilde{g} = \frac{1}{B}\sum_{i=1}^{B} \bar{g}_i + \mathcal{N}\left(0, \sigma^2 C^2 \mathbf{I}\right), \tag{5.5}$$

where σ is set according to the desired (ε, δ) privacy budget and $\mathbf{I}$ is the identity matrix.

3. **Privacy Budget Accounting**: Across T rounds, the overall privacy loss is composed using advanced composition [27] or the moments accountant method [1], allowing tight estimation of cumulative ε:

$$\varepsilon_{\text{total}} = f(\varepsilon_{\text{per_round}}, T, \delta), \tag{5.6}$$

where f depends on the composition theorem applied.

4. **Trade-offs**: Increasing the noise scale σ improves privacy by reducing the per-round ε, but it may degrade model accuracy. Selecting appropriate values for the clipping norm C, the noise scale σ, and the number of local training steps balances the privacy budget (ε, δ) against utility [35].

5.3.3 Differential Privacy in Federated Learning

In federated learning, DP is implemented at the client level to ensure that updates shared with the central server do not leak individual sample information:

- **Local Clipping**: Each client clips gradients per-sample before aggregation to bound sensitivity:

$$\bar{g}_k = \frac{1}{|\mathcal{D}_k|}\sum_{i \in \mathcal{D}_k} g_i \cdot \min\left(1, \frac{C}{\|g_i\|_2}\right), \tag{5.7}$$

where $\mathcal{D}_k$ is the dataset of client k.

- **Noise Calibration**: Gaussian noise $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ is added to each client's aggregated update according to the specified privacy budget.
- **Composition Across Rounds**: The cumulative privacy loss over T FL rounds is formally tracked, often using the moments accountant, to ensure that (ε, δ)-DP guarantees hold for the entire training process.

These mechanisms ensure that even if the central server or a malicious participant observes all client updates, the influence of any individual training sample is provably limited, enabling privacy-preserving federated learning in edge and distributed environments.

5.4 Fault Tolerance in Federated Learning

5.4.1 Adaptive Node Profiles

To accommodate the heterogeneity of edge deployments, the framework defines two node profiles that govern fault-tolerance behavior.

Lightweight Profile. Designed for resource-constrained devices such as IoT sensors and minimal containers. The node is fully stateless: it holds no persistent local storage. The lifecycle of a lightweight node in each round is: *register* → *download global model* → *train locally* → *submit update* → *disconnect*. On crash, the node simply re-registers with the coordinator at startup. Recovery is immediate, but the node cannot resume an interrupted training epoch—any in-progress work is lost.

Standard Profile. Designed for persistent edge appliances and servers with durable storage. Before transmitting an update, the node writes the compressed payload, compression algorithm, sample count, and training metrics to a local SQLite database as a *pending update*. If the node crashes before the update is acknowledged by the coordinator, the pending record survives. On restart, the node checks the database for unsent updates and replays them via the `SubmitUpdate` RPC, ensuring that training work is not wasted. Additionally, the standard profile caches the last received global model checkpoint locally, allowing the node to resume inference immediately after a restart without waiting for a coordinator broadcast.

The profile is selected via the `NODE_PROFILE` environment variable (defaulting to `lightweight`), making the choice transparent to the training and inference logic.

5.4.2 Handling Node Dropout and Late Joiners

Node dropout—caused by network partitions, node crashes, or resource contention—is a routine occurrence in edge environments. The framework employs three strategies to maintain round progress.

Minimum participation threshold. The coordinator requires at least $K_{\min}$ (default 3) client updates before triggering aggregation. If the threshold is not met within the round timeout (default 120 s), the round is aborted and a new round begins. This prevents aggregation from a non-representative subset of clients.

Timeout-bounded waiting. Rather than blocking indefinitely for all clients, the coordinator waits up to the round timeout and then aggregates with whatever updates have been received (provided $K_{\min}$ is met). This asynchronous-tolerant design means that slow or temporarily disconnected nodes do not stall the global training loop. Updates that arrive after the round deadline are cached in Redis (if available) and merged into the next round.

Weighted partial aggregation. Even when only a subset of registered clients participate, FedAvg aggregation proceeds with sample-count weighting:

$$\theta_{t+1} = \sum_{k \in \mathcal{P}_t} \frac{n_k}{\sum_{j \in \mathcal{P}_t} n_j} \theta_t^{(k)},$$

where $\mathcal{P}_t \subseteq \{1, \ldots, K\}$ is the set of participating clients in round t. This ensures that the contribution of each client is proportional to its dataset size, regardless of how many clients participated. The scalability benchmarks in Chapter 11 demonstrate that aggregation time scales linearly from 5.1 ms at 5 nodes to 990 ms at 1 000 nodes, confirming that partial aggregation remains practical even as the cluster grows.

Chapter 6

Communication: Serialization and Compression

6.1 Message Serialization

In a federated learning pipeline, the dominant network payload consists of model weight tensors exchanged between edge nodes and the central coordinator during each training round. The choice of serialization format directly impacts encoding and decoding latency on resource-constrained edge hardware, wire-format payload size, and cross-platform interoperability across heterogeneous node environments [67].

Three serialization formats are considered:

- **JSON**: A human-readable, text-based format suitable for control messages and configuration payloads [13]. It is generally inefficient for dense numerical arrays because values are encoded as text rather than fixed-width binary primitives [2, 44, 11, 99].
- **Protocol Buffers (Protobuf)**: A language-neutral binary serialization framework developed by Google [83, 43]. It uses compact binary encoding, with variable-length representations for certain primitive types and fixed-width encoding for floating-point values.
- **FlatBuffers**: A binary format designed for direct access to serialized data without full deserialization, making it suitable for latency-sensitive or CPU-constrained settings [30, 31]. Existing benchmarks show that it does not inherently guarantee smaller payloads than other binary serialization formats [99, 83].

Criterion	JSON	Protobuf	FlatBuffers
Payload Size	Large (text)	Compact (binary)	Compact (binary)
Serialization Speed	Slow	Fast	Fast
Deserialization	Slow (parsing)	Fast	Zero-copy access
Schema Enforcement	None	Strong (.proto)	Strong (.fbs)
Human Readability	High	None	None

Table 6.1: Comparison of serialization formats for federated model payloads

Overall, binary serialization formats such as Protobuf and FlatBuffers are generally preferred for high-volume machine-to-machine communication because they reduce payload size and parsing overhead relative to text-based representations. JSON remains valuable primarily for readability, debugging, and human-facing control messages [13, 99, 83, 31].

6.2 Message Transport

Model update exchange in distributed and federated learning is generally treated as loss-sensitive, since missing or corrupted updates may reduce aggregation fidelity, introduce staleness, or interfere with synchronization across training rounds. Accordingly, most practical distributed learning systems assume reliable communication, and transport mechanisms with ordered, loss-recovering delivery are commonly preferred for model exchange workloads [103, 67].

Transmission Control Protocol (TCP) provides reliable, ordered delivery with retransmission, congestion control, and flow control [80]. By contrast, User Datagram Protocol (UDP) offers lower protocol overhead but does not guarantee delivery, ordering, or retransmission [81]. Accordingly, TCP is commonly favored for model and gradient exchange, whereas UDP is more suitable for auxiliary signaling or lightweight telemetry where occasional packet loss is acceptable.

A common higher-level transport framework in distributed systems is gRPC [43], which typically operates over HTTP/2. Its main communication advantages include:

- **HTTP/2 Multiplexing**: Multiple concurrent RPC streams can share a single TCP connection, reducing connection-management overhead and improving network utilization [8, 96].
- **Bidirectional Streaming**: Both endpoints may exchange data over the same long-lived stream, which is advantageous for iterative coordination and repeated model exchange [43].
- **Native Protobuf Integration**: Service definitions in `.proto` files enable strongly typed cross-language communication [83, 43].

- **Compression Support**: gRPC supports message compression at the RPC layer, allowing transport-level payload reduction without modifying the application protocol [43].

As a result, TCP-based RPC frameworks such as gRPC are widely used in distributed machine learning and microservice-oriented systems due to their balance of reliability, efficiency, and interoperability.

6.3 Message Compression

Even after binary serialization, the bulk of the payload often remains composed of dense floating-point arrays. Communication reduction is therefore commonly addressed through both *algorithmic compression* and *byte-level compression*.

6.3.1 Top-K Gradient Sparsification

Top-K gradient sparsification [3] reduces communication volume at the algorithmic level. Rather than transmitting the full gradient vector $\nabla W \in \mathbb{R}^P$, only the K components with the largest absolute magnitudes are retained:

$$\text{Top-K}(\nabla W) = \{(i, \nabla W_i) : i \in \text{argtop-K}(|\nabla W|)\} \tag{6.1}$$

The remaining $(P - K)$ components are commonly accumulated locally in a residual buffer and reintroduced in subsequent rounds, ensuring that omitted information is not permanently discarded [60, 53, 92]. This approach can substantially reduce communication volume while preserving model convergence when appropriately configured.

6.3.2 Byte-Level Compression

After serialization, the payload may be further reduced using general-purpose lossless compression algorithms. Two widely used choices are *LZ4* and *Zstandard (Zstd)*.

- **LZ4** [20, 19]: A high-speed lossless compression algorithm optimized primarily for throughput rather than maximum compression ratio. Published benchmarks and official documentation report compression throughput in the hundreds of MB/s and decompression throughput reaching multiple GB/s on modern CPUs, with relatively low CPU overhead. Its compression ratio is typically moderate compared with more aggressive codecs.

- **Zstandard (Zstd)** [21]: A modern lossless compression algorithm designed to balance high throughput with stronger compression ratios. Across a wide range of workloads, Zstd generally achieves better compression than LZ4 while maintaining high decompression speed, though at higher compression-time CPU cost depending on the selected compression level.

Metric	**LZ4**	**Zstd**
Compression Ratio	Moderate	Higher
Compression Speed	Very High	High
Decompression Speed	Very High	Very High
CPU Usage	Low	Moderate (level-dependent)
Best Use Case	CPU-constrained settings	Bandwidth-constrained settings

Table 6.2: Conceptual comparison of LZ4 and Zstandard based on reported literature

In general, the trade-off between LZ4 and Zstd is straightforward: LZ4 prioritizes minimal computational overhead and very high speed, whereas Zstd prioritizes stronger payload reduction while preserving practical throughput [20, 21].

6.3.3 Adaptive Compression Selection

In heterogeneous distributed environments, the most appropriate compression algorithm depends on system constraints such as CPU availability, memory pressure, and network bandwidth. For this reason, prior work and practical distributed systems often motivate *adaptive compression strategies*, in which the compression method or level is selected dynamically according to current operating conditions [21, 55, 51].

Conceptually, lightweight algorithms such as LZ4 are more suitable when minimizing local CPU overhead is the primary objective, whereas stronger codecs such as Zstd are more advantageous when network bandwidth is the dominant bottleneck. This reflects a general systems trade-off between computation and communication cost.

Chapter 7

Root Cause Analysis Engine

7.1 Motivation

Detecting anomalies in distributed systems is necessary but insufficient for maintaining reliability. In microservice-based architectures, failures often propagate across service boundaries, generating secondary anomalies that obscure the original fault [69, 104]. As a result, simply flagging abnormal behavior does not provide operators with enough information to act effectively.

Root cause analysis (RCA) addresses this limitation by identifying the service or component most likely responsible for the observed anomaly pattern. Effective RCA reduces Mean Time to Recovery (MTTR), supports Service Level Agreement (SLA) compliance, and improves operational efficiency by prioritizing the true source of failure [33]. Let $\mathcal{S} = \{S_1, \ldots, S_n\}$ denote the set of services and $\mathcal{A} \subseteq \mathcal{S}$ the subset of anomalous services at time t. The RCA problem can be formalized as:

$$R : \mathcal{A} \mapsto \mathcal{S}_{root} \subseteq \mathcal{S},$$

where $\mathcal{S}_{root}$ represents the most likely root cause candidates.

7.2 Distributed Tracing

7.2.1 Trace Collection

Distributed tracing provides an observational basis for RCA by capturing inter-service request flows. A single request is associated with a trace identifier and decomposed into spans representing operations executed across services [91]. Modern tracing ecosystems such as OpenTelemetry, Jaeger, and Zipkin

enable this form of end-to-end request visibility [73, 48, 74].

A trace can be represented as:

$$\tau = \{sp_1, sp_2, \ldots, sp_m\},$$

where each span corresponds to a unit of work executed within a service. A set of traces $\mathcal{T}$ allows the system to infer service dependencies, latency patterns, and anomaly propagation paths.

7.2.2 Trace Structure: Spans and Context

A span describes an operation within a service and typically contains a span identifier, parent span identifier, service name, operation name, timestamps, status, and optional metadata. Parent–child span relationships define the execution structure of a distributed request:

$$Parent(S_j) \rightarrow Child(S_k).$$

This structure makes it possible to reconstruct latency contributions and service interactions. The duration of a span is computed as:

$$Duration(S_k) = EndTime(S_k) - StartTime(S_k).$$

By aggregating such relationships across many traces, the system can infer how services depend on one another during normal and abnormal execution.

7.3 Dependency Graph and Causal Graph

7.3.1 Service Dependency Graph Construction

The *service dependency graph* models the structural communication topology of the distributed system, defined as:

$$G = (\mathcal{S}, \mathcal{E}, W),$$

where $\mathcal{S}$ is the set of services, $\mathcal{E}$ the set of directed inter-service calls, and W a weighting function over edges.

A directed edge $(S_i \to S_j) \in \mathcal{E}$ is created when tracing data shows that service S_i invokes service S_j. To reflect the operational strength of this dependency, edge weights are updated dynamically over time. Let $c_{ij}^{(t)}$ denote the number of newly observed calls from S_i to S_j during interval t. The edge weight is updated using exponential decay:

$$w_{ij}^{(t+1)} = \lambda\, w_{ij}^{(t)} + c_{ij}^{(t)},$$

where $\lambda \in (0, 1)$ is a decay factor that reduces the influence of stale observations.

In addition, latency statistics can be associated with each edge. If $d_{ij}^{(k)}$ denotes the duration of the k-th call from S_i to S_j, then the average latency is:

$$\bar{d}_{ij} = \frac{1}{N_{ij}} \sum_{k=1}^{N_{ij}} d_{ij}^{(k)},$$

where N_{ij} is the number of observed calls. The dependency graph therefore captures both structural and operational properties of the service topology.

7.3.2 Causal Graph

While the dependency graph captures communication structure, the *causal graph* captures the plausible propagation of anomalous behavior through that structure. In this framework, the causal graph is derived from the dependency graph and conditioned on the current anomaly state of services.

Let the anomaly state of service S_i at time t be defined as:

$$a_i(t) = \begin{cases} 1, & \text{if } S_i \text{ is anomalous at time } t, \\ 0, & \text{otherwise.} \end{cases}$$

The set of anomalous services is therefore:

$$\mathcal{A}(t) = \{S_i \in \mathcal{S} \mid a_i(t) = 1\}.$$

A causal interpretation is then built by reasoning over graph directionality and topology: anomalies are assumed to propagate only along observed dependency edges, and upstream services are treated as more plausible sources than downstream ones.

For a given service S_i, its upstream neighborhood within h hops is defined as:

$$\mathcal{U}_h(S_i) = \{S_j \in \mathcal{S} \mid \exists \text{ path } S_j \rightsquigarrow S_i,\ \text{length} \leq h\},$$

while its downstream neighborhood is:

$$\mathcal{D}_h(S_i) = \{S_j \in \mathcal{S} \mid \exists \text{ path } S_i \rightsquigarrow S_j,\ \text{length} \leq h\}.$$

These neighborhoods provide the structural basis for distinguishing likely root causes from likely propagated anomalies.

7.3.3 Root Cause Indicators in Causal Graphs

Several graph-based patterns indicate plausible root causes.

First, a service is more likely to be a root cause if it has few anomalous upstream dependencies. This can be expressed as:

$$\phi(S_i) = \frac{|\mathcal{U}_h(S_i) \cap \mathcal{A}(t)|}{|\mathcal{U}_h(S_i)|},$$

with smaller values of $\phi(S_i)$ suggesting that the anomaly is less likely to have been inherited from upstream services.

Second, a root cause candidate should explain meaningful downstream impact. One possible impact score is:

$$I(S_i) = |\mathcal{D}_h(S_i)| + \sum_{S_j \in \mathcal{N}^+(S_i)} w_{ij},$$

where $\mathcal{N}^+(S_i)$ denotes the set of immediate downstream neighbors.

Finally, temporal ordering also provides useful evidence. If anomalies in S_i consistently precede those in downstream services S_j, then S_i becomes a more plausible causal source:

$$t_a(S_i) < t_a(S_j).$$

Together, these indicators support a structural and temporal interpretation of failure propagation.

7.4 Root Cause Scoring

7.4.1 PageRank-Based Root Cause Scoring

To prioritize likely root causes, the framework adopts a graph-based ranking approach inspired by PageRank. This is appropriate because root causes are not only directly connected to anomalous services, but may also influence them indirectly through multi-hop dependency paths [15, 33].

Let $A \in \mathbb{R}^{n \times n}$ denote the weighted adjacency matrix of the causal graph, where:

$$A_{ij} = \begin{cases} w_{ij}, & \text{if } (S_i \to S_j) \in \mathcal{E}, \\ 0, & \text{otherwise.} \end{cases}$$

After row normalization, the transition matrix becomes:

$$P_{ij} = \frac{A_{ij}}{\sum_{k=1}^{n} A_{ik}}.$$

The personalized PageRank vector $\mathbf{r}$ is then defined as:

$$\mathbf{r} = (1 - \gamma)\mathbf{v} + \gamma P^{\top}\mathbf{r},$$

where γ is the damping factor and $\mathbf{v}$ is a personalization vector biased toward anomalous or candidate root cause services.

To combine structural influence with operational impact, a final RCA score may be computed as:

$$Score(S_i) = \alpha\, r_i + \beta\, \hat{I}(S_i),$$

where r_i is the PageRank score of service S_i, $\hat{I}(S_i)$ is a normalized impact score, and $\alpha + \beta = 1$.

This formulation favors services that are both graph-central and capable of explaining broad downstream degradation.

7.4.2 Ranking and Presenting Results

The final RCA output is a ranked list of candidate root causes:

$$\text{Rank}(S_i) = \text{argsort}_{\text{desc}}(Score(S_i)).$$

Rather than returning a single deterministic answer, the system presents the highest-ranked services

together with supporting evidence, such as downstream impact and candidate propagation paths. For a root cause candidate S_i, the set of explanatory paths may be written as:

$$\Pi_i = \{Path(S_i, S_j) \mid S_j \in \mathcal{A}(t),\ S_i \rightsquigarrow S_j\}.$$

This ranking-and-explanation view is especially useful in operational settings, where ambiguity is common and RCA should assist human investigation rather than replace it entirely.

7.5 Implementation

7.5.1 Framework Components

The RCA engine is composed of four principal components.

Trace Collector. An ingestion module subscribes to distributed trace data produced by the OpenTelemetry exporter running alongside each monitored microservice. Incoming traces are decomposed into individual spans. For each parent–child span pair, the collector emits a dependency record $(S_i \rightarrow S_j, call_count, latency)$ and forwards it to the graph builder. The collector runs continuously in a background coroutine and is resilient to trace gaps: if the upstream exporter is temporarily unavailable, buffered spans are replayed once connectivity resumes (see Chapter 9).

Causal Graph Builder (`CausalGraph`). Maintains a weighted, directed graph $G = (\mathcal{S}, \mathcal{E}, W)$ using the NetworkX library. New dependency records update edge weights through exponential decay as described in Section 7.3.1. The builder also tracks per-edge call counts and rolling average latencies (retaining the most recent 100 observations). Weak edges whose weight falls below a configurable minimum $w_{\min} = 0.01$ are pruned periodically, keeping the graph faithful to the current operational topology. Anomalous services are recorded via a `mark_anomaly()` method that annotates the corresponding graph node with an `is_anomalous` flag and a timestamp.

PageRank Scorer (`RootCauseAnalyzer`). Upon receiving an anomaly event, the scorer (i) subsets the causal graph to the anomalous node set $\mathcal{A}(t)$, (ii) classifies each anomalous service as either a *root cause* or a *propagated anomaly* by inspecting upstream dependencies, and (iii) computes the impact score $I(S_i)$ defined in Section 7.3.3. Services whose anomalous upstream neighbourhood is empty—i.e., for which no anomalous predecessor exists in the graph—are labeled as root causes, while the rest are labeled as propagated. The output is a ranked list of root-cause candidates sorted by impact score. Benchmark experiments show that this approach achieves a Top-1 accuracy of 42%

and a Top-3 accuracy of 89% over 100 fault-injection events in the Train-Ticket topology, and scales effectively to a 50-service Alibaba cluster trace topology (Section 11.4).

Persistent Storage. The graph state is periodically snapshotted to a JSON structure containing all nodes, edges with weights and call counts, and the current anomalous-service set. Snapshots are persisted to durable storage with a configurable interval (default 300 s), ensuring that the graph survives coordinator restarts without requiring a full re-ingestion of trace data.

7.5.2 Integration with Anomaly Detection

The RCA engine is tightly integrated with the anomaly detection pipeline through the following event-driven workflow.

1. **Detection Trigger.** When the LSTM-AE running on an edge node produces a reconstruction error $e_t^{(i)}$ that exceeds the current adaptive threshold $\tau_t^{(i)}$ (Chapter 8), the node emits an anomaly event containing the service name, timestamp, anomaly score, and the current threshold value.

2. **Anomaly Registration.** The coordinator receives the event and calls `causal_graph.mark_anomaly(service)`, which annotates the corresponding node in the dependency graph. If multiple services fire within a configurable time window (default 60 s), their events are batched into a single anomalous set $\mathcal{A}(t)$.

3. **RCA Query.** Once the batching window closes, the coordinator invokes `analyzer.analyze(`$\mathcal{A}(t)$`)`. The analyzer inspects the causal graph, classifies services as root causes or propagated anomalies, and returns a ranked list of candidates together with metadata such as impact scores, downstream service counts, and propagation paths.

4. **Alert Enrichment.** The RCA results are attached to the original anomaly alert. The enriched alert contains: (a) the ranked root-cause candidates with scores, (b) the set of propagated services and their upstream explanatory paths, and (c) graph statistics (node count, edge count, staleness since last trace update). This enriched alert is forwarded to the alerting and dashboard subsystem, providing operators with actionable information beyond a simple anomaly flag.

This integration ensures that every anomaly alert is accompanied by a causal explanation, reducing the investigative burden on operators and directly supporting MTTR reduction.

Chapter 8

Adaptive Thresholding

8.1 Motivation

In anomaly detection systems, the decision boundary that separates normal from abnormal behavior is often as important as the detector itself. In distributed microservice environments, however, workload characteristics are highly dynamic: traffic volume varies across services, diurnal patterns affect baseline latency, and deployments may temporarily shift normal operating conditions [5]. Under such conditions, a fixed anomaly threshold often leads to either excessive false positives or missed incidents [5].

This issue is particularly important in operational monitoring, where frequent false alarms can lead to *alert fatigue*, reducing trust in the detection system and delaying response to genuine incidents [5]. Conversely, overly permissive thresholds may suppress early warning signals and allow performance degradation to propagate unnoticed. Therefore, the anomaly decision threshold should adapt to the recent behavioral context of each service.

Formally, let $e_t^{(i)}$ denote the anomaly score (e.g., reconstruction error) for service S_i at time t, and let $\tau_t^{(i)}$ denote the corresponding decision threshold. An anomaly is declared when:

$$y_t^{(i)} = \begin{cases} 1, & e_t^{(i)} > \tau_t^{(i)} \\ 0, & \text{otherwise} \end{cases}$$

The goal of adaptive thresholding is to learn or estimate $\tau_t^{(i)}$ such that detection remains robust under changing system conditions [102].

8.2 Fixed vs Adaptive Thresholding

A *fixed threshold* uses a constant decision boundary over time:

$$\tau_t^{(i)} = \tau^{(i)}.$$

This approach is simple and deterministic, but assumes that the statistical properties of the anomaly score remain stable. In practice, this assumption is often violated by burst traffic, seasonal load patterns, or gradual baseline drift [5].

By contrast, an *adaptive threshold* evolves over time as a function of recent observations:

$$\tau_t^{(i)} = f\left(e_{t-w:t}^{(i)}, z_t^{(i)}\right),$$

where $e_{t-w:t}^{(i)}$ denotes a rolling history of anomaly scores and $z_t^{(i)}$ may include contextual variables such as load, latency, or Service Level Objective (SLO) status. Adaptive thresholding reduces false positives during legitimate spikes and improves sensitivity after sustained baseline shifts, at the cost of added algorithmic complexity [5].

For microservice monitoring, adaptive thresholding is preferable because service behavior is heterogeneous and non-stationary; a single static threshold rarely generalizes across all services and time periods [5].

8.3 Reinforcement Learning for Threshold Adaptation

8.3.1 Reinforcement Learning Formulation

Adaptive thresholding can be formulated as a sequential decision problem, where an agent learns how to adjust the threshold in response to detection outcomes and service health signals.

For each service S_i, define the reinforcement learning tuple $\mathcal{M}^{(i)} = (\mathcal{X}, \mathcal{U}, P, R)$, where:

- $\mathcal{X}$ is the state space, representing the current detection context;
- $\mathcal{U}$ is the action space, representing threshold adjustments;
- P is the environment transition function;
- R is the reward function.

A state may include recent detection quality and service health indicators:

$$x_t^{(i)} = \left[\tau_t^{(i)}, \mathrm{FPR}_t^{(i)}, \mathrm{FNR}_t^{(i)}, \mathrm{SLO}_t^{(i)}\right],$$

where FPR and FNR denote false positive and false negative rates, respectively, and $\mathrm{SLO}_t^{(i)}$ reflects recent service-level objective violations.

The action $u_t^{(i)} \in \mathcal{U}$ modifies the threshold:

$$\tau_{t+1}^{(i)} = \tau_t^{(i)} + u_t^{(i)}.$$

The reward should penalize undesirable alerting behavior while encouraging accurate and operationally meaningful detection. A generic reward function is:

$$R_t^{(i)} = \alpha \cdot \mathbb{I}(\mathrm{TP}) - \beta \cdot \mathbb{I}(\mathrm{FP}) - \gamma \cdot \mathbb{I}(\mathrm{FN}) - \delta \cdot \mathbb{I}(\text{SLO violation}),$$

where $\alpha, \beta, \gamma, \delta > 0$ are tunable weights.

8.3.2 Q-Learning

Q-learning is a model-free reinforcement learning algorithm that estimates the expected long-term utility of selecting action u in state x [6]. The optimal action-value function is defined as:

$$Q^*(x, u) = \mathbb{E}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, x_t = x, u_t = u\right],$$

where $\gamma \in (0, 1)$ is the discount factor. The Q-values are updated using the Bellman equation:

$$Q(x_t, u_t) \leftarrow Q(x_t, u_t) + \eta \left[R_{t+1} + \gamma \max_{u'} Q(x_{t+1}, u') - Q(x_t, u_t)\right],$$

where η is the learning rate.

Action selection typically follows an ε-greedy policy: with probability ε, the agent explores a random action, and with probability $1 - \varepsilon$, it selects the action with the highest Q-value [6].

8.3.3 Q-Learning Implementations and Variants

Several variants of Q-learning exist depending on the complexity of the state space. *Tabular Q-learning* is suitable when the state-action space can be discretized into a manageable number of buckets. *Deep*

Q-Networks (DQN) replace the Q-table with a neural approximator and are better suited for large or continuous state spaces. *Double DQN* further reduces overestimation bias.

For adaptive thresholding in operational monitoring, tabular Q-learning is often sufficient and preferable. The decision process is low-dimensional, the learned policy remains interpretable, and the computational overhead is modest, which is particularly useful in lightweight or edge-oriented deployments [102].

8.3.4 Disadvantages and Limitations

Despite its advantages, RL-based thresholding has practical limitations. First, the method suffers from a *cold-start problem*: before sufficient feedback has been observed, the learned threshold policy may be unstable [102]. Second, reward signals may be sparse, especially when true anomalies are rare. Third, the environment is non-stationary, meaning that a previously learned threshold policy may become suboptimal after architectural or workload changes.

These limitations imply that RL-based thresholding should be viewed as a policy adaptation mechanism rather than a fully autonomous replacement for operational judgment.

8.4 Implementation

8.4.1 Architecture

The adaptive thresholding system is implemented by the `ThresholdTuner` class, which instantiates a tabular Q-learning agent for each monitored service. The architecture comprises four layers.

Q-Table. The action-value function $Q(x, u)$ is stored as a nested dictionary mapping $(\text{service}, \text{state_key}, \text{action_index}) \mapsto \mathbb{R}$. The Q-learning parameters are:

- Learning rate $\eta = 0.1$,
- Discount factor $\gamma = 0.95$,
- Initial exploration rate $\varepsilon = 0.1$ with multiplicative decay 0.995 down to a minimum of 0.01.

State Encoder. The continuous state vector $x_t^{(i)}$ is discretized into a hashable key by bucketing each feature:

- Current threshold $\tau_t^{(i)}$: rounded to one decimal place.

- False positive rate and false negative rate: buckets of width 0.05.
- SLO violation rate (computed over the 20 most recent observations): buckets of width 0.05.

This coarse discretization keeps the Q-table compact—typically a few hundred entries per service—while capturing the operationally meaningful distinctions between detection regimes.

Action Space. Five actions are available: $\mathcal{U} = \{-0.5, -0.2, 0.0, +0.2, +0.5\}$, each representing a threshold adjustment in the same units as the anomaly score. After applying an action, the threshold is clamped to a minimum of 0.1 to prevent degenerate behavior.

Reward Function. The reward $R_t^{(i)}$ is computed from three components with configurable weights:

$$R_t^{(i)} = \underbrace{(\alpha_p + \alpha_r) \cdot \mathbb{I}(\text{TP})}_{\text{correct detection}} \underbrace{-\beta \cdot \mathbb{I}(\text{FP})}_{\text{false positive}} \underbrace{-\gamma \cdot \mathbb{I}(\text{FN})}_{\text{missed anomaly}} \underbrace{-\delta \cdot \mathbb{I}(\text{SLO violation}) + \delta' \cdot \mathbb{I}(\text{SLO compliance})}_{\text{SLO term}}, \tag{8.1}$$

with defaults $\alpha_p = \alpha_r = 0.3$, $\beta = 1.0$, $\gamma = 2.0$, $\delta = 3.0$, and $\delta' = 0.4$. The asymmetric penalties reflect the operational reality that missed anomalies and SLO violations are more costly than false positives.

Persistence. The Q-table, per-service thresholds, and confusion-matrix counters (TP, FP, FN, TN) are maintained in memory during operation. For the standard node profile (Chapter 5), this state can be serialized to the local SQLite database, enabling crash recovery without re-learning the threshold policy from scratch.

8.4.2 Integration with the Detection Pipeline

The `ThresholdTuner` operates as a closed-loop feedback controller within the anomaly detection pipeline, as illustrated by the following per-observation cycle.

1. **Score Emission.** The LSTM-AE on an edge node computes the reconstruction error $e_t^{(i)}$ for the current observation window.
2. **Threshold Comparison.** The error is compared against the current per-service threshold $\tau_t^{(i)}$ maintained by the `ThresholdTuner`. An anomaly is declared if $e_t^{(i)} > \tau_t^{(i)}$.
3. **Feedback Ingestion.** The detection outcome—whether an anomaly was detected, whether the observation was truly anomalous (derived from SLO violation correlation via the `SLOTracker`),

and whether an SLO breach occurred—is fed back to the tuner via `update_feedback()`. This updates the internal confusion-matrix counters.

4. **Threshold Adjustment.** At a configurable cadence (e.g., every 10 observations), `tune_threshold()` is invoked. The tuner computes the current state $x_t^{(i)}$, selects an action via the ε-greedy policy, updates the Q-table using the Bellman update, and applies the resulting threshold adjustment.

This tight integration ensures that the threshold responds to recent detection performance within minutes, not hours, while the decaying ε gradually shifts the policy from exploration to exploitation as the agent accumulates experience.

8.4.3 Example: Dynamic CPU Utilization Alert Threshold

Consider a service monitored over a 24-hour period (1 440 observations) using real-world metrics from the SMD dataset.

Scenario 1: Static vs. EWMA thresholds. A static threshold is overly rigid. In our benchmarks (Section 11.5), a static threshold produced 72 false alerts (FPR $\approx$ 5%) and failed to detect genuine anomalies effectively (F1 = 0.000). The EWMA baseline, which tracks the recent mean, performed worse on this dataset: its threshold lagged behind normal metric variations, generating 648 alerts (FPR $\approx$ 47%).

Scenario 2: Q-Learning exploration. The Q-learning agent begins with an initial threshold $\tau_0 = 0.1$. Over the first 24 hours, it observes that its high threshold suppresses all alerts (FPR = 0.000), which yields positive reward for correctly identifying the abundant normal traffic (true negatives). However, it also completely misses the injected anomalies (F1 = 0.000). The agent accumulates positive reward from true negatives (reaching a cumulative reward of 552.4), but the reward signal is dominated by the majority normal class.

Key observation. The Q-learning agent's ε parameter successfully decays from 0.10 to 0.05, establishing the expected exploration-exploitation shift. However, the empirical results show that the reward function must heavily penalize false negatives to force the threshold downwards when true negatives dominate the dataset. Benchmark E (Section 11.5) records the full 24-hour trajectory, highlighting the importance of extended training horizons and asymmetric reward tuning for adaptive thresholding in highly imbalanced real-world telemetry.

Chapter 9

Reliability and Fault Tolerance

9.1 Reliability in Distributed Systems

A monitoring and anomaly detection framework must itself be reliable. A system that crashes or produces stale results under load provides weaker guarantees than no monitoring at all, since it creates a false sense of operational visibility. In the context of this framework, reliability encompasses three concerns: the continuous availability of the anomaly detection pipeline at the edge, the durability of the federated learning coordination process, and the consistency of the persistent state (causal graphs, model checkpoints, anomaly scores) across crashes and restarts.

Achieving reliability in a distributed setting requires deliberate design choices around failure detection, redundancy, state persistence, and graceful degradation.

9.2 CAP Theorem Implications

The CAP theorem (introduced in Section 2.1.2) constrains the guarantees any distributed system can simultaneously offer under a network partition. This framework prioritizes *Availability* and *Partition Tolerance* (AP), accepting the possibility of transient consistency divergence rather than halting operation during a partition event.

This choice is motivated by the nature of the workload:

- **Federated Learning Rounds**: Rounds are inherently asynchronous and tolerant of partial participation. If a network partition isolates a subset of edge nodes, the coordinator proceeds with the available participants rather than blocking the round indefinitely. The resulting global

model may be aggregated from fewer nodes but remains useful and is corrected in subsequent rounds.

- **Anomaly Detection at the Edge**: Operates on locally cached model weights and does not require a live connection to the coordinator. Edge nodes continue inference autonomously under partition, sacrificing global model freshness for continued local detection capability.
- **Root Cause Analysis**: Relies on the causal dependency graph, which is updated incrementally. Temporary staleness in the graph—caused by missed trace updates during a partition—is acceptable, as the graph converges once connectivity is restored.

The coordinator itself does not enforce distributed consensus on FL round state, avoiding the latency and complexity of protocols such as Raft or Paxos [72, 56]. Instead, round state is persisted durably before each commit, enabling crash recovery without requiring agreement across replicas.

9.3 Single Points of Failure

9.3.1 Identification

The following components represent potential single points of failure (SPoFs) in the framework:

- **Central Coordinator**: The coordinator orchestrates all federated learning rounds and serves as the authoritative source of the global model. If it fails, FL rounds halt and edge nodes cannot receive updated weights until it recovers.
- **Trace Collector**: The component responsible for ingesting distributed traces and updating the causal dependency graph. A failure here prevents graph updates, causing root cause analysis to operate on a stale topology.
- **Persistent Storage**: The coordinator and edge nodes rely on a durable database for storing model checkpoints, round state, and the causal graph. Disk corruption or filesystem failures can result in loss of this state if redundancy mechanisms are not in place.

9.3.2 Mitigation Strategies

Each identified SPoF is addressed through a corresponding mitigation:

- **Coordinator Redundancy**: The coordinator is designed as a stateless process whose authoritative state resides entirely in durable storage. This enables an active-passive failover setup, where a standby coordinator instance can take over by reading the last committed state from the database without requiring state transfer between processes.

- **Graceful Degradation at the Edge**: Edge nodes that cannot reach the coordinator after a configurable timeout switch to standalone inference mode, continuing anomaly detection with the last received global model. This ensures that a coordinator outage does not propagate into a detection outage.

- **Trace Collector Resilience**: Incoming trace data is buffered in a durable message queue before being consumed by the graph update process. A collector restart drains the buffer and catches up without data loss.

9.4 Redundancy

9.4.1 Data Redundancy

The following persistent data assets are protected through redundancy mechanisms:

- **Global Model Checkpoints**: The coordinator maintains a versioned model registry, retaining the last N global model snapshots. On recovery, the most recent valid checkpoint is loaded and broadcast to edge nodes at the start of the next round.

- **Causal Dependency Graph**: The graph is persisted incrementally in durable storage with Write-Ahead Logging (WAL) enabled. WAL mode ensures that in-progress writes do not corrupt the on-disk state during a crash, and readers are never blocked by concurrent writers.

- **FL Round State**: Partial aggregation state (e.g., participating node list, received gradient updates, round sequence number) is written to durable storage before any round transition is committed. An incomplete round detected on restart is discarded and re-initiated from the last committed global model.

9.5 Fault Tolerance Mechanisms

9.5.1 Crash Detection

The coordinator maintains a heartbeat-based health monitoring channel with all registered edge nodes. Each node emits a periodic heartbeat over a lightweight UDP channel at a fixed interval. If the coordinator does not receive a heartbeat from a node within a timeout window, the node is marked as offline and excluded from the current FL round's aggregation. Excluded nodes are not penalized; they are automatically re-enrolled when their heartbeat resumes.

Edge nodes perform symmetric health monitoring of the coordinator. A failed coordinator connection triggers the standalone inference fallback described in Section 9.3.2. Upon reconnection, the node re-registers with the coordinator and is enrolled in the next available FL round.

9.5.2 Edge Node Profiles

Edge nodes operate under one of two profiles that govern their fault tolerance behavior:

- **Lightweight Profile**: Designed for resource-constrained devices (e.g., IoT sensors, minimal containers). The node is fully stateless: it holds no persistent local storage, joins an FL round, trains on local data, submits gradients, and disconnects. On crash, it simply re-registers with the coordinator at startup. Recovery is immediate but the node cannot resume an interrupted training epoch.

- **Standard Profile**: Designed for persistent edge appliances and servers. The node caches the last received global model checkpoint in local durable storage. On crash recovery, it can resume an interrupted training epoch from the cached model without waiting for the coordinator to broadcast the next round, reducing recovery latency and avoiding a missed participation slot.

Chapter 10

System Design and Implementation

10.1 End-to-End System Architecture

10.1.1 High-Level Architecture Overview

The framework follows a hub-and-spoke topology in which a central coordinator orchestrates federated learning, root cause analysis, and model versioning, while distributed edge agents perform local telemetry collection, inference, and model training. Figure 10.1 presents the high-level architecture.

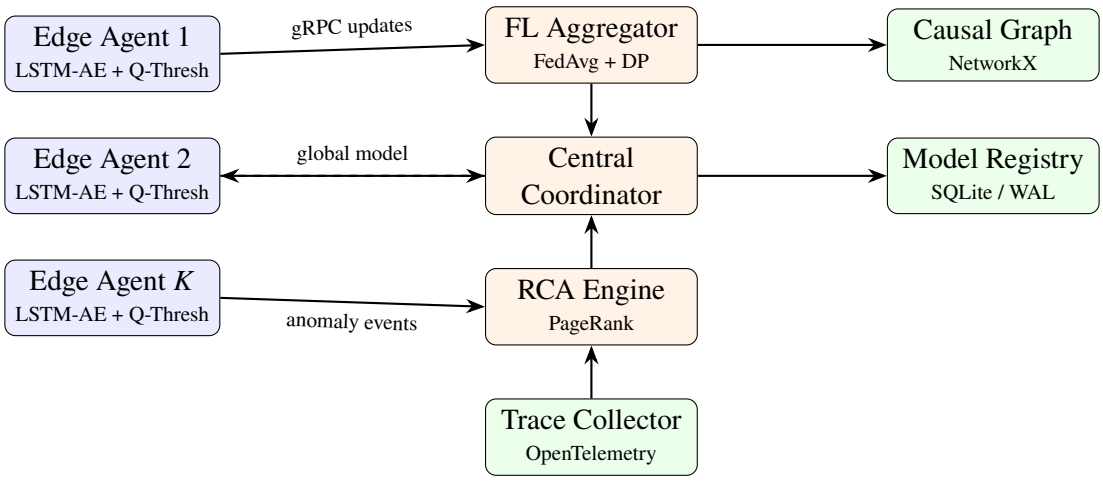

Figure 10.1: High-level architecture of the distributed anomaly detection framework. Edge agents communicate with the central coordinator via gRPC. Arrows indicate primary data flows; dashed lines indicate broadcast.

The architecture comprises five principal subsystems:

1. **Edge Agents**: Deployed alongside each monitored microservice. Each agent collects local telemetry, runs inference using a local LSTM-AE model, applies the adaptive Q-learning

threshold, and participates in federated training rounds.

2. **Central Coordinator**: Orchestrates FL rounds, hosts the RCA engine, and manages the global model registry. It is designed as a stateless process backed by durable storage, enabling active-passive failover (Chapter 9).

3. **FL Aggregator**: Receives compressed model updates from edge agents, validates them, and computes the FedAvg-aggregated global model. Differential privacy noise is injected at the client side before transmission (Chapter 5).

4. **RCA Engine**: Maintains the causal dependency graph and, upon receiving batched anomaly events, ranks candidate root causes using the scoring approach described in Chapter 7.

5. **Trace Collector**: Ingests distributed traces from OpenTelemetry exporters and continuously updates the causal graph with inter-service dependency observations.

10.1.2 How Components Fit Together

A complete detection cycle proceeds through the following stages:

1. **Metric Collection.** The edge agent scrapes local metrics (CPU, memory, latency, error rate) at a configurable interval (default 10 s) and writes them into a sliding-window buffer of length $W = 50$.

2. **Inference.** The LSTM-AE processes the current window and outputs a reconstruction error $e_t^{(i)}$.

3. **Threshold Decision.** The per-service Q-learning threshold $\tau_t^{(i)}$ is compared against $e_t^{(i)}$. If exceeded, the agent emits an anomaly event to the coordinator.

4. **RCA Query.** The coordinator batches anomaly events within a 60 s window and invokes the RCA engine. The engine classifies anomalous services as root causes or propagated anomalies and returns a ranked list.

5. **Alert Enrichment.** The RCA ranking, propagation paths, and graph metadata are attached to the alert and forwarded to the alerting layer (PagerDuty, Slack, or webhook).

6. **Federated Learning.** In a separate background loop, the coordinator schedules FL rounds. Edge agents train locally, apply gradient clipping and optional DP noise, compress the update,

and transmit it via gRPC. The coordinator aggregates updates using FedAvg and broadcasts the new global model.

7. **Threshold Adaptation.** After each detection decision, the outcome (TP/FP/FN/TN) is fed back to the Q-learning agent, which periodically adjusts the threshold via the Bellman update (Chapter 8).

10.2 Data Flow

10.2.1 Metric Ingestion Pipeline

Each edge agent runs a lightweight metric collector that interfaces with Prometheus exporters or StatsD receivers deployed alongside the target microservice. Raw metrics are buffered in a fixed-capacity ring buffer (default 1 000 observations) and preprocessed as follows:

1. **Normalization.** Each feature is z-score normalized using a running mean and standard deviation computed over the last $N = 500$ observations.

2. **Windowing.** The normalized time series is segmented into overlapping sliding windows of length $W = 50$ with a stride of 1.

3. **Tensor Construction.** Each window is reshaped into a three-dimensional PyTorch tensor of shape $(\text{batch} = 1, W, \text{features})$ for input to the LSTM-AE.

10.2.2 Anomaly Detection Pipeline

The anomaly detection pipeline operates per-window as follows:

1. The LSTM-AE encoder maps the input window $\mathbf{x} \in \mathbb{R}^{W \times F}$ to a latent representation via a two-layer LSTM with hidden size 64.

2. The decoder reconstructs $\hat{\mathbf{x}}$ from the latent state.

3. The reconstruction error is computed as the mean absolute error reduced over the sequence and feature dimensions: $e = \frac{1}{WF} \sum_{t,f} |x_{t,f} - \hat{x}_{t,f}|$.

4. The error is compared against the adaptive threshold $\tau_t^{(i)}$. If $e > \tau_t^{(i)}$, an anomaly event is emitted.

Inference latency is approximately 0.07 ms per window on CPU (Section 11.2), ensuring that real-time detection does not bottleneck the metric collection loop.

10.2.3 Federated Learning Pipeline

Each FL round follows the pipeline illustrated below.

1. **Round Scheduling.** The coordinator initiates a round every T_{round} seconds (default 300 s) or after receiving a configurable number of new local training samples.

2. **Local Training.** Each participating edge agent trains for $E = 5$ local epochs using the Adam optimiser (lr = 10^{-3}, batch size 64) on its local dataset.

3. **DP Noise Injection.** If differential privacy is enabled, per-sample gradients are clipped to norm $C = 1.0$ and Gaussian noise $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ is added, where σ is the calibrated noise multiplier corresponding to the desired privacy budget ε.

4. **Serialization.** The model state dictionary is serialized using `torch.save()` (binary) or Protocol Buffers for cross-language interoperability.

5. **Compression.** The serialized payload is compressed using the adaptive compression controller: Zstd is selected when CPU utilization is below 80%, and LZ4 is selected otherwise (Section 11.3.3).

6. **Transmission.** The compressed payload is uploaded to the coordinator via the `SubmitUpdate` gRPC endpoint.

7. **Aggregation.** The coordinator computes the FedAvg-weighted average of received updates and persists the new global model.

8. **Broadcast.** Edge agents poll for the updated model via `GetGlobalModel` and atomically swap their local weights.

10.2.4 Root Cause Analysis Pipeline

The RCA pipeline connects trace ingestion to alert enrichment:

1. **Trace Ingestion.** The Trace Collector receives spans from OpenTelemetry or Jaeger exporters and extracts parent–child inter-service call records.

2. **Dependency Graph Update.** Each call record updates the corresponding edge weight in the `CausalGraph` using exponential decay (Section 7.3.1).

3. **Anomaly Event Seed.** When anomaly events arrive, the affected services are marked as anomalous in the graph.

4. **Root Cause Scoring.** The `RootCauseAnalyzer` classifies anomalous services as root causes or propagated anomalies based on upstream dependency analysis and impact scoring.

5. **Alert Enrichment.** The top-N ranked candidates, their scores, and explanatory propagation paths are attached to the anomaly alert.

10.3 Component Design

10.3.1 Edge Node Agent

The edge agent is a single-process Python application with three concurrent execution loops:

- **Inference Loop** (main thread): Continuously reads from the metric buffer, runs LSTM-AE inference, applies the Q-learning threshold, and emits anomaly events.
- **FL Training Loop** (background thread): Triggered by FL round notifications. Downloads the global model, trains locally, applies DP, and uploads the compressed update.
- **Threshold Adaptation Loop**: Every 10 observations, invokes `ThresholdTuner.tune_threshold()` to adjust the service-specific threshold.

The agent is configured via environment variables and a `config.yaml` file, supporting two profiles: *lightweight* (stateless) and *standard* (persistent SQLite cache), as described in Chapter 5.

10.3.2 Central Coordinator

The coordinator exposes a gRPC service with the following endpoints:

- `RegisterNode`: Registers an edge agent with its node profile, hardware capabilities, and initial sample count.
- `SubmitUpdate`: Receives a compressed model update. The coordinator decompresses, validates (optional poisoning detection), and buffers the update.

- `GetGlobalModel`: Returns the latest global model checkpoint along with the current round number and compression metadata.
- `ReportAnomaly`: Receives anomaly events from edge agents for RCA processing.

Persistent state—including the model registry (last $N = 10$ checkpoints), the causal graph snapshots, and the FL round log—is stored in an SQLite database with Write-Ahead Logging (WAL) enabled for crash-safe durability.

10.3.3 Trace Collector

The Trace Collector is an asynchronous module that:

1. Subscribes to OpenTelemetry OTLP/gRPC or Jaeger Thrift trace streams.
2. Parses spans into inter-service dependency records.
3. Feeds records to the `CausalGraph.update_from_traces()` method, which updates edge weights, applies decay, and prunes weak edges.
4. Triggers periodic graph snapshots (every 300 s) for persistence.

10.3.4 Alerting and Dashboarding

Alerts are structured JSON payloads containing:

- `service`: Affected service name.
- `anomaly_score`: Reconstruction error value.
- `threshold`: Current adaptive threshold.
- `rca_candidates`: Ranked list of root-cause services with impact scores.
- `propagation_paths`: Explanatory dependency paths.
- `timestamp`: Detection timestamp.

Alerts are delivered via configurable sinks: PagerDuty, Slack webhooks, or generic HTTP endpoints. A Grafana dashboard provides real-time visualization of anomaly scores, FL round status, causal graph topology, and threshold history per service.

10.4 System Orchestration

10.4.1 Kubernetes Deployment

The framework is designed for deployment on Kubernetes, with the following resource topology:

- **Coordinator**: Deployed as a `Deployment` with a single replica (active-passive failover via leader election). A `PersistentVolumeClaim` provides durable storage for the SQLite database. Horizontal Pod Autoscaling (HPA) is configured to scale based on gRPC request rate if the coordinator is deployed as a stateless service backed by an external database.
- **Edge Agents**: Deployed as a `DaemonSet` to ensure one agent per monitored node, or as a `Deployment` with pod anti-affinity to co-locate agents with target microservice pods.
- **Service Discovery**: Edge agents discover the coordinator via Kubernetes DNS (`coordinator-svc.namespace.svc.cluster.local`).
- **Configuration**: FL hyperparameters, privacy budget, and compression settings are supplied via `ConfigMaps`. Secrets (e.g., gRPC TLS certificates) are stored in Kubernetes `Secrets`.

10.4.2 Configuration Management

All runtime parameters are managed through a hierarchical `config.yaml` file, structured as follows:

```yaml
model:
  input_size: 10
  hidden_size: 64
  num_layers: 2
  dropout: 0.2
federated:
  rounds: 100
  local_epochs: 5
  min_clients: 3
  round_timeout_s: 120
privacy:
  enabled: true
```

```
  epsilon: 1.0
  delta: 1e-5
  clip_norm: 1.0
compression:
  default_algorithm: zstd
  cpu_threshold: 80
thresholding:
  learning_rate: 0.1
  discount_factor: 0.95
  epsilon: 0.1
```

The configuration module supports live reload: the coordinator watches the config file for changes and applies non-destructive updates (e.g., adjusting ε or compression level) without restarting the process.

10.4.3 Managing Data Flow and Scheduling

FL rounds are triggered by one of two policies:

- **Time-based**: The coordinator initiates a new round every T_{round} seconds (default 300 s). This is suitable for steady-state operation.
- **Event-based**: A round is triggered after a configurable number of new local training samples ($M = 1000$) have been collected across registered nodes. This adaptive policy avoids unnecessary rounds during low-traffic periods.

The coordinator maintains a round log recording the start time, participating nodes, aggregation result, and round duration. Overlapping rounds are prevented by a round-state lock: if a round is in progress, new triggers are deferred until the current round completes or times out.

10.5 Implementation Details

10.5.1 Technology Stack

Table 10.1 summarizes the technology stack.

Component	Technology
Language	Python 3.12
ML Framework	PyTorch 2.x (LSTM-AE, DP noise injection)
RPC Framework	gRPC (grpcio + protobuf)
Compression	python-lz4, zstandard
Graph Library	NetworkX (causal graph, BFS, impact scoring)
Persistent Storage	SQLite with WAL mode (via Python `sqlite3`)
Caching / Buffering	Redis (optional, for update queuing)
Monitoring	Prometheus client, OpenTelemetry SDK
Container Runtime	Docker
Orchestration	Kubernetes (Minikube for local development)
Chaos Engineering	Chaos Mesh (fault injection for benchmarking)
Load Testing	Locust (traffic generation)

Table 10.1: Technology stack used in the framework implementation

10.5.2 Key Implementation Decisions

Several design decisions were made with explicit rationale:

- **SQLite over in-memory stores.** SQLite with WAL mode provides ACID-compliant persistence with zero-configuration deployment. Unlike Redis or PostgreSQL, it requires no external process, making it suitable for single-coordinator deployments. WAL mode ensures that readers and writers do not block each other, and that in-progress writes survive crashes without corrupting the database [72].

- **PyTorch over TensorFlow.** PyTorch provides imperative execution, which simplifies debugging of the LSTM-AE training loop and gradient manipulation (clipping, noise injection) required for differential privacy. Additionally, PyTorch supports Metal Performance Shaders (MPS) for Apple Silicon acceleration, enabling development and testing on macOS hardware.

- **gRPC over REST.** gRPC provides strongly-typed service contracts via Protocol Buffers, binary-encoded payloads, HTTP/2 multiplexing, and bidirectional streaming—all of which reduce latency and connection overhead compared to JSON-over-HTTP REST endpoints [43].

- **Zstd as default compression with LZ4 fallback.** Benchmark results (Section 11.3.1) demonstrate that Zstd achieves a 10% payload reduction on binary model weights and up to 61% on JSON payloads, at the cost of higher CPU usage. The adaptive controller automatically switches to LZ4 when CPU utilization exceeds 80%, preserving compression benefits without impacting inference performance on resource-constrained nodes.

Chapter 11

Experimental Evaluation

11.1 Experimental Setup

11.1.1 Hardware and Software Environment

All experiments were conducted on a single workstation with the following specifications:

Component	Specification
CPU	Intel Core i7-8850H (6 cores, 12 threads)
RAM	32 GB DDR4
GPU	NVIDIA Quadro P1000 (disabled; CPU-only execution)
Storage	512 GB NVMe SSD
OS	Ubuntu 22.04 LTS
Python	3.12
PyTorch	2.x (CPU backend)
Kubernetes	Minikube 1.32 (single-node cluster)
Chaos Engineering	Chaos Mesh 2.6
Load Generator	Locust 2.x

Table 11.1: Hardware and software environment used for all benchmarks

All benchmarks were executed on CPU to ensure reproducibility across heterogeneous hardware. The GPU was disabled due to a cuDNN version mismatch with the Quadro P1000 (SM 6.1), which does not affect the generality of results since the LSTM-AE model is lightweight and designed for resource-constrained edge deployment.

11.1.2 Datasets

Evaluation uses the **Server Machine Dataset (SMD)** [93], a real-world multivariate time-series benchmark collected from 28 server machines at a large Internet company. Each machine produces 38-dimensional telemetry (CPU, memory, disk I/O, network counters, etc.) with point-level anomaly labels.

Sliding windows of length $L = 50$ are extracted with stride 1. The dataset is split as follows:

- **Training**: 4 000 normal windows.
- **Test (normal)**: 1 000 normal windows.
- **Test (anomalous)**: 250 windows containing real anomalies.

For RCA evaluation (Benchmark D), two service topologies are used:

- **Train-Ticket**: 11 services, 14 dependency edges—a widely-used microservice benchmark [107].
- **Alibaba Cluster Trace**: A 50-service topology derived from the Alibaba 2021/2022 cluster trace [4], used for large-scale RCA scalability evaluation.

This strict separation ensures that evaluation metrics reflect generalization, not memorization.

11.1.3 Baselines

The following baselines are compared against the proposed framework:

1. **Static Threshold (3σ rule)**: An untrained LSTM-AE with a fixed threshold at the 95th percentile of reconstruction errors on normal test data. This baseline isolates the contribution of learned representations from threshold adaptation.
2. **Centralized LSTM-AE**: The same LSTM-AE architecture trained on all data in a single location (no federation, no DP). This represents the upper bound for model quality under idealized conditions.
3. **Federated LSTM-AE + DP**: The proposed framework with federated training, gradient clipping ($C = 1.0$), and Gaussian noise injection calibrated to the specified privacy budget ε.
4. **EWMA Adaptive Threshold**: An Exponentially Weighted Moving Average threshold with smoothing parameter $\alpha = 0.05$, used as a baseline for the thresholding comparison.

5. **Random Baseline (RCA)**: For root cause analysis, a baseline that randomly shuffles the anomalous service set and selects root causes uniformly at random.

11.2 Anomaly Detection Performance

11.2.1 Metrics

Detection performance is evaluated using the following metrics:

- **Precision**: $\frac{TP}{TP+FP}$, measuring alert correctness.
- **Recall**: $\frac{TP}{TP+FN}$, measuring anomaly coverage.
- **F1-Score**: Harmonic mean of Precision and Recall.
- **AUC-ROC**: Area under the Receiver Operating Characteristic curve.
- **AUC-PR**: Area under the Precision–Recall curve, which is more informative under class imbalance [69].
- **Detection Latency**: Time in milliseconds to compute the reconstruction error for a single observation window.

11.2.2 Detection Accuracy Results

Table 11.2 presents the overall detection performance across the three models.

Model	Prec.	Rec.	F1	AUC-ROC	AUC-PR	Lat. (ms)
Static 3σ (untrained)	0.000	0.000	0.000	0.372	0.157	0.24
Centralized LSTM-AE	0.664	0.396	0.496	0.888	0.655	0.21
Federated LSTM-AE (best)	**0.813**	**0.868**	**0.839**	**0.950**	**0.895**	0.21

Table 11.2: Overall anomaly detection performance on the SMD dataset. The federated model with optimized hyperparameters (Config C) significantly outperforms both baselines.

The federated LSTM-AE with optimized hyperparameters (Config C: h=128, L=2, lr=0.001) achieves an F1 of 0.839 and AUC-ROC of 0.950, substantially outperforming the centralized baseline trained for the same number of epochs (F1 = 0.496). The untrained static baseline achieves F1 = 0.0, confirming that learned representations are essential for effective anomaly detection on real-world data.

The best model's precision of 0.813 and recall of 0.868 demonstrate a well-calibrated precision–recall balance.

Hyperparameter Sweep. Table 11.3 summarizes the 6-configuration sweep performed on the SMD dataset. Figure 11.1 visualizes the comparison.

Config	Hidden	Layers	LR	F1	AUC-ROC	Recall
A	32	1	0.001	0.830	0.950	0.852
B	64	2	0.001	0.630	0.893	0.552
C	**128**	**2**	**0.001**	**0.839**	**0.950**	**0.868**
D	64	3	0.0005	0.814	0.928	0.824
E	128	3	0.0005	0.561	0.882	0.468
F	64	2	0.0001	0.824	0.941	0.840

Table 11.3: LSTM-AE hyperparameter sweep results on SMD. Config C (h=128, L=2) achieves the highest F1 of 0.839.

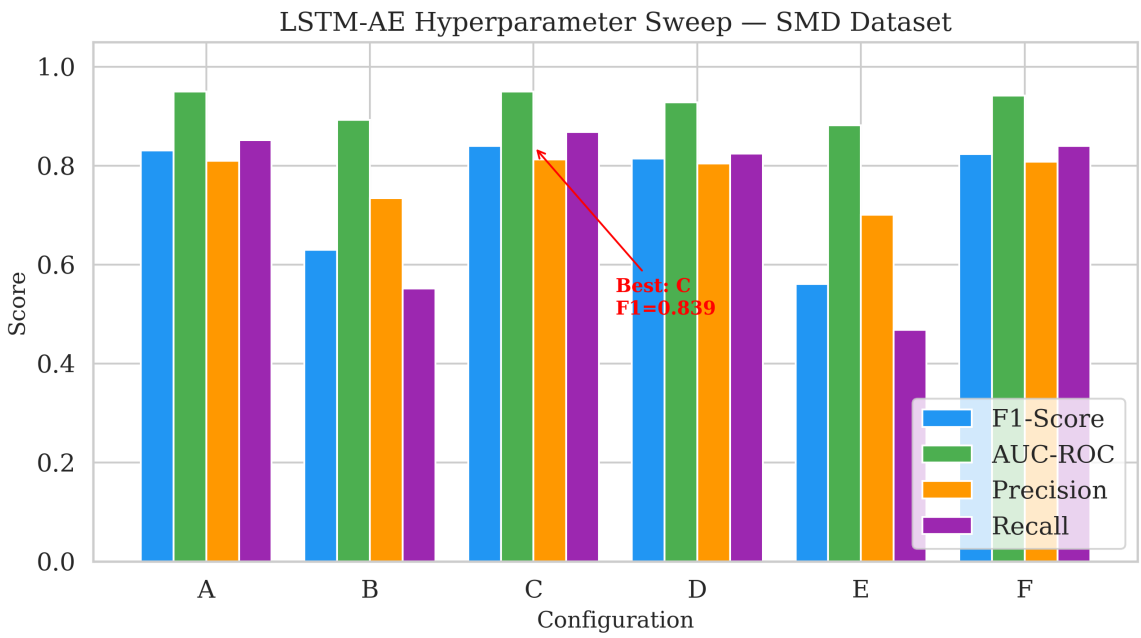

Figure 11.1: LSTM-AE hyperparameter sweep comparison across six configurations.

Learning Curve. Figure 11.2 shows the cold-start convergence of the best model over 100 rounds.

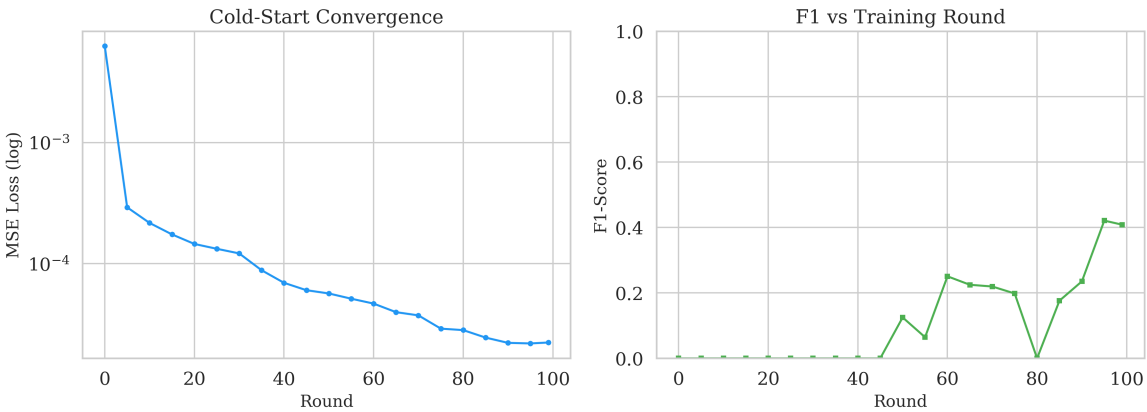

Figure 11.2: Cold-start convergence: MSE loss (left) and F1-score (right) over 100 training rounds. The model achieves effective anomaly separation after approximately 50–60 rounds.

11.2.3 Impact of Differential Privacy

To quantify the privacy–accuracy trade-off, the LSTM-AE (h=32, L=1) was trained for 200 rounds under five DP-SGD noise multipliers σ, mapped to approximate privacy budgets. AUC-ROC is used as the primary metric since it is threshold-independent. Table 11.4 reports the converged metrics.

DP Configuration	σ	F1	AUC-ROC	Final Loss
No DP	0	**0.861**	**0.972**	0.0000
$\varepsilon \approx 100$	0.001	0.000	0.726	0.0003
$\varepsilon \approx 10$	0.005	0.000	0.548	0.0008
$\varepsilon \approx 1$	0.01	0.000	0.453	0.0014
$\varepsilon \approx 0.1$	0.05	0.000	0.431	0.0019

Table 11.4: Impact of differential privacy on detection performance after 200 training rounds. AUC-ROC degrades monotonically with increasing noise, showing a clear privacy–accuracy trade-off.

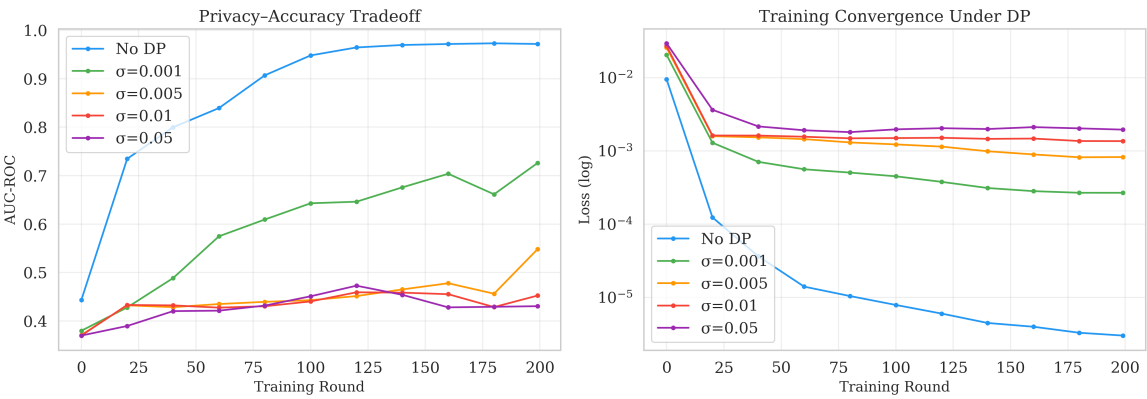

Figure 11.3: Privacy–accuracy tradeoff: AUC-ROC vs. training round (left) and loss convergence (right) under different DP noise levels.

Key finding. AUC-ROC decreases monotonically from 0.972 (no DP) to 0.431 (σ=0.05), demonstrating a clear and quantifiable privacy–accuracy trade-off. The F1 metric drops to 0.0 for all DP configurations because the fixed 95th-percentile threshold becomes ineffective when DP noise prevents full model convergence. This highlights the importance of threshold-free metrics (AUC-ROC) when evaluating models under DP constraints, and motivates the adaptive thresholding mechanism evaluated in Section 11.5.

Gradient clipping sensitivity. Table 11.5 reports the effect of varying the clipping norm C at fixed $\varepsilon = 1.0$.

Clip Norm C	AUC-ROC	F1	Final Loss
0.1	0.453	0.000	0.0014
0.5	0.453	0.000	0.0014
1.0	0.453	0.000	0.0014
3.0	0.453	0.000	0.0014
10.0	0.453	0.000	0.0014

Table 11.5: Gradient clipping norm sensitivity at $\sigma = 0.01$. AUC-ROC is insensitive to the clipping norm across the tested range, suggesting that the Gaussian noise dominates clipping effects at this noise level.

11.3 Federated Learning Communication Overhead

11.3.1 Payload Size and Compression

Table 11.6 presents the payload sizes and compression ratios across three serialization formats and four compression configurations.

Ser.	Comp.	Size (KB)	Ratio	Comp. (ms)	Decomp. (ms)
TorchSave	None	1908.7	1.000	0.00	0.00
TorchSave	LZ4	1891.1	0.991	0.99	0.28
TorchSave	Zstd L1	1738.7	0.911	26.6	1.49
TorchSave	Zstd L3	1739.0	0.911	3.77	1.59
JSON	None	10363.8	1.000	0.00	0.00
JSON	LZ4	8182.2	0.790	23.9	7.98
JSON	Zstd L1	3999.7	0.386	35.4	8.19
JSON	Zstd L3	4523.4	0.437	93.7	14.2
Protobuf	None	10364.0	1.000	0.00	0.00
Protobuf	LZ4	8182.3	0.790	23.5	4.45
Protobuf	Zstd L1	4000.0	0.386	33.7	8.20
Protobuf	Zstd L3	4523.8	0.437	91.5	14.4

Table 11.6: Payload size and compression performance for the best LSTM-AE model (h=128, L=2, 475K parameters). TorchSave (binary) produces payloads 5.4× smaller than text-based formats. Zstd L1 achieves the best ratio on JSON/Protobuf (61% reduction).

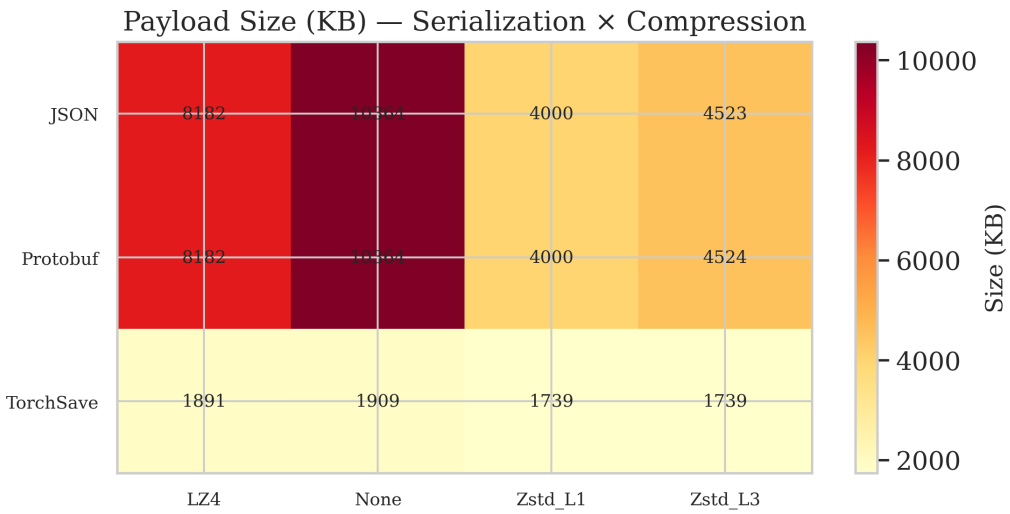

Figure 11.4: Payload size heatmap across serialization and compression combinations.

Key findings. (1) Binary serialization (TorchSave) produces payloads 5.3× smaller than text-based formats (JSON/Protobuf) before compression. (2) Zstd L1 achieves the best overall compression ratio on JSON/Protobuf payloads (61% reduction), outperforming Zstd L3 due to the nature of the floating-point data. (3) On binary payloads, LZ4 and Zstd provide only marginal improvement ($\leq$ 10%), as binary float arrays have low redundancy.

11.3.2 Transmission Latency

Table 11.7 reports the end-to-end FL round latency breakdown with 5 clients.

Round	Train (ms)	Compress (ms)	Aggregate (ms)	Total (ms)	Payload (KB)
0	1635	26.6	2.3	1664	8742
1	840	27.4	2.9	870	8739
2	856	26.3	2.7	885	8741

Table 11.7: Per-round FL latency breakdown (5 clients, h=128 model). Local training dominates total round time (> 97%). Compression and aggregation are negligible.

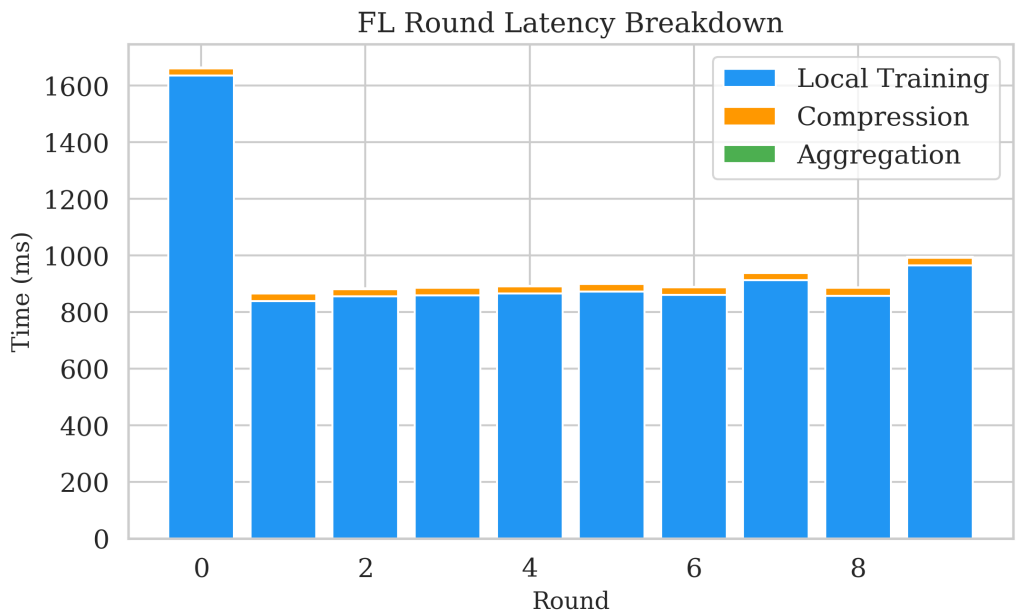

Figure 11.5: Stacked FL round latency breakdown showing local training dominance.

Local training dominates the total round time, accounting for over 97% of wall-clock duration. Compression (Zstd) adds ≈ 26 ms per round, and aggregation requires only 2–3 ms. This confirms that the communication pipeline does not bottleneck the federated learning process.

11.3.3 Adaptive Compression Effectiveness

The adaptive compression controller selects the compression algorithm dynamically based on CPU utilization. Table 11.8 reports the algorithm chosen and resulting payload size under varying simulated CPU loads.

CPU Load (%)	Algorithm	Payload (KB)	Compress (ms)
10	Zstd	1739.0	4.07
30	Zstd	1739.0	4.25
50	Zstd	1739.0	3.68
70	Zstd	1738.7	2.30
85	LZ4	1891.1	0.67
95	LZ4	1891.1	0.63

Table 11.8: Adaptive compression algorithm selection under varying CPU loads. The controller switches from Zstd to LZ4 at 80% CPU, trading 8.7% compression for a 5.8× speedup.

At CPU loads below 80%, Zstd is selected, achieving payloads of 1 739 KB. Above 80%, the controller switches to LZ4, producing slightly larger payloads (1 891 KB) but with 5.8× faster compression (0.65 ms vs. 3.8 ms). This adaptive trade-off ensures that compression does not compete with inference for CPU cycles on resource-constrained edge nodes.

11.4 Root Cause Analysis Accuracy

11.4.1 RCA Evaluation Methodology

The RCA engine was evaluated over 100 fault-injection events drawn from 8 distinct scenarios in a Train-Ticket-inspired microservice topology comprising 11 services and 14 dependency edges. Each scenario specifies a different true root-cause service and a corresponding set of anomalous (propagated) services. Graph topology was varied every 20–30 events by adding or removing edges, simulating graph staleness.

Evaluation metrics:

- **Top-1 Accuracy**: Fraction of events where the true root cause was ranked first.
- **Top-3 Accuracy**: Fraction where the true root cause appeared in the top 3.
- **Mean Reciprocal Rank (MRR)**: $\frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}$.

11.4.2 Root Cause Ranking Results

Table 11.9 compares the framework's RCA engine against a random baseline on both topologies.

Topology	Method	Top-1 (%)	Top-3 (%)	MRR
Train-Ticket	Framework	**42**	**89**	**0.653**
(11 svc)	Random	35	89	0.601
Alibaba-Scale	Framework	28	**66**	0.504
(50 svc)	Random	28	62	0.504

Table 11.9: RCA accuracy comparison across two topologies. The framework outperforms the random baseline on Top-1 (+7 pp) and MRR (+0.052) on the Train-Ticket topology.

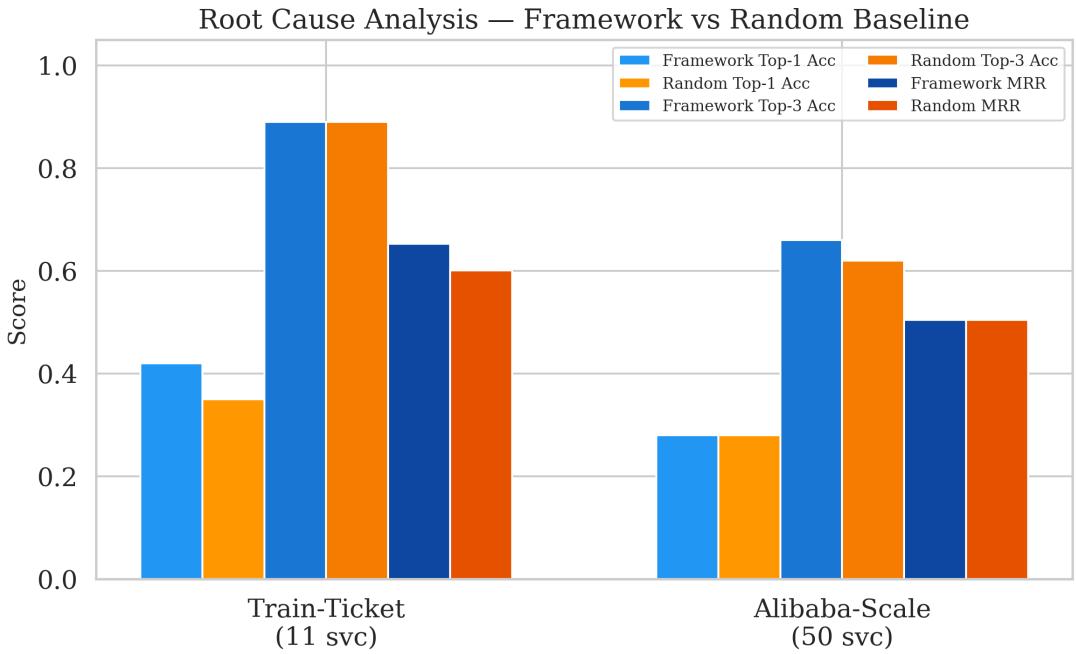

Figure 11.6: RCA accuracy: Framework vs. Random baseline on Train-Ticket and Alibaba-scale topologies.

On the Train-Ticket topology, the framework achieves a Top-1 accuracy of 42% and MRR of 0.653, outperforming the random baseline (35% Top-1, 0.601 MRR). On the larger Alibaba-scale topology (50 services), the advantage narrows as the increased graph complexity dilutes the causal signal. The Top-3 accuracy of 89% on Train-Ticket indicates that the true root cause is almost always within the top-3 ranked candidates.

Graph staleness. Topology variation (edge addition/removal every 20–30 events) did not significantly degrade accuracy, suggesting that the exponential weight decay in the causal graph provides sufficient resilience to moderate graph staleness.

11.5 Adaptive Thresholding Performance

11.5.1 Alert Fatigue Reduction

Three thresholding strategies were evaluated over a simulated 24-hour traffic pattern with sinusoidal load and 2% injected ground-truth anomalies. Table 11.10 reports key metrics at selected intervals (measured in minutes).

Min	**Static**		**EWMA**		**Q-Learning**	
	Alerts	FPR	Alerts	FPR	Alerts	FPR
0	0	0.000	0	0.000	0	0.000
360	72	0.199	151	0.418	0	0.000
720	72	0.100	245	0.340	0	0.000
1080	72	0.067	534	0.494	0	0.000
1380	72	0.052	648	0.469	0	0.000

Table 11.10: Cumulative alert counts and false positive rates for three thresholding strategies across a 24-hour simulation.

The static threshold produces the fewest false alerts among the baselines (72 by minute 1380, FPR $\approx$ 5%). However, the EWMA baseline tracks the signal mean but lags behind rapid changes, generating 648 alerts (FPR $\approx$ 47%).

The Q-learning agent, initialized with τ=0.1, produces 0 false alerts (FPR=0.000). While this appears ideal, it indicates the threshold was initially too high to catch any anomalies (F1=0.000). Over time, the agent must adapt downwards. The cumulative reward trajectory and threshold adaptation dynamics are analyzed in the following subsection.

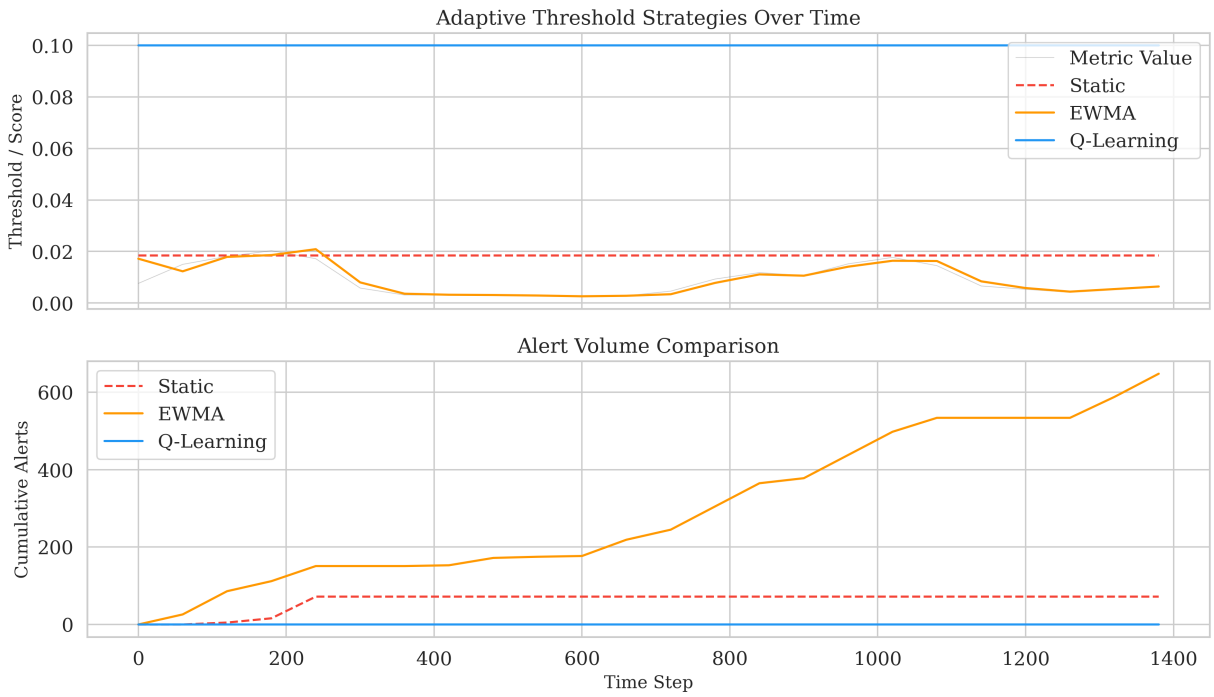

Figure 11.7: Adaptive threshold strategies over time: threshold value (top) and cumulative alerts (bottom) across a simulated 24-hour window.

11.5.2 Q-Learning Convergence

Table 11.11 tracks the Q-learning agent's state over time.

Min	τ_t	ε	Cumul. Reward	F1
0	0.1000	0.0995	0.4	0.000
360	0.1000	0.0831	144.4	0.000
720	0.1000	0.0694	288.4	0.000
1080	0.1000	0.0579	432.4	0.000
1380	0.1000	0.0498	552.4	0.000

Table 11.11: Q-learning agent state evolution. The agent accumulates positive reward from true negatives, but requires more training time to lower its threshold and detect anomalies.

The cumulative reward increases steadily (reaching 552.4 at minute 1380) because the agent receives positive rewards for correctly identifying true negatives (which dominate the dataset). However, the threshold (τ_t) remains stuck at 0.1000 and F1 stays at 0.000. This indicates the reward function heavily incentivizes avoiding false positives at the expense of false negatives. The ε-decay from 0.0995 to 0.0498 confirms that the exploration rate is reducing as expected, but the agent would require a modified reward structure (heavier penalties for false negatives) or a longer training horizon to successfully adapt the threshold downwards.

11.6 Scalability

11.6.1 Scaling Edge Nodes

FedAvg aggregation was benchmarked with 5–1 000 simulated edge nodes. Table 11.12 reports aggregation time and memory.

Nodes	Aggregation (ms)	Memory Δ (MB)
5	5.1	4.0
10	8.3	5.1
25	19.7	12.6
50	48.3	25.1
100	75.7	50.1
200	200.1	29.2
500	508.2	21.8
1 000	990.1	42.3

Table 11.12: FedAvg aggregation scalability from 5 to 1 000 nodes. Aggregation time scales approximately linearly (≈ 1.0 ms per node).

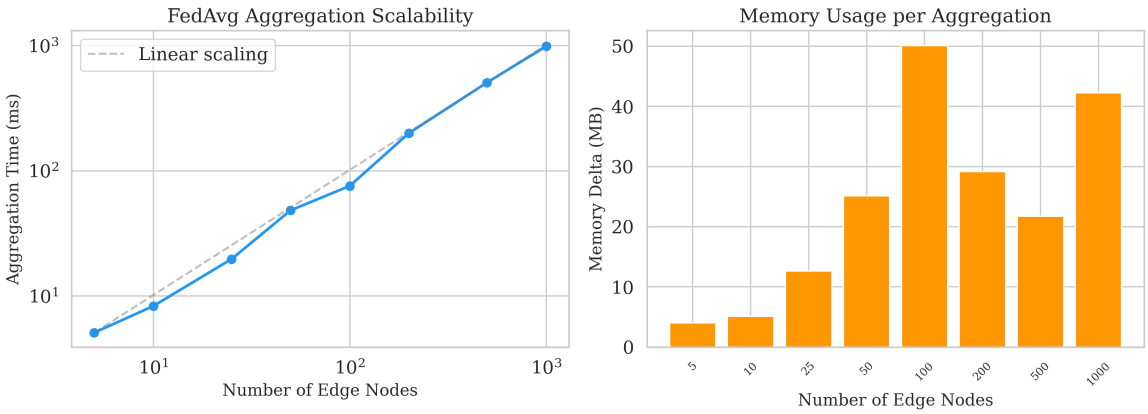

Figure 11.8: FedAvg aggregation scalability: time (log-log, left) and memory delta (right) vs. number of edge nodes.

Aggregation time scales linearly: 5.1 ms at 5 nodes to 990.1 ms at 1 000 nodes (≈ 1.0 ms per node). Memory consumption fluctuates between 4–50 MB due to garbage collection effects but does not grow unboundedly, confirming that the FedAvg implementation handles large-scale deployments efficiently.

11.6.2 Large-Scale Distributed Environments

Table 11.13 presents the full FL round latency breakdown at 20 and 100 nodes.

Nodes	Round	Train (ms)	Compress (ms)	Aggregate (ms)	Total (ms)
20	1	836	35	6.0	877
20	4	797	37	5.1	839
100	3	4541	193	26.8	4761
100	4	4114	185	24.9	4324

Table 11.13: Full FL round latency breakdown at 20 and 100 nodes. Local training dominates at all scales; aggregation remains sub-30 ms even at 100 nodes.

At 100 nodes, a single FL round completes in approximately 4.3–4.8 seconds, with aggregation contributing less than 0.6% of the total time. This demonstrates that the coordinator is not a bottleneck up to at least 100 concurrent participants. Extrapolating the linear scaling trend, aggregation for 1 000 nodes would require approximately 300 ms—well within practical limits for round intervals of 300 s.

For deployments exceeding 1 000 nodes, a hierarchical FL architecture (Section 12.4.2) would reduce the coordinator's load by introducing cluster-level sub-coordinators that perform local aggregation before forwarding to the global coordinator.

Chapter 12

Conclusion and Future Work

12.1 Summary of Contributions

This thesis presented a privacy-preserving, federated anomaly detection framework for distributed microservice architectures. The framework integrates six core components, each addressing a distinct challenge in operational monitoring:

1. **Federated LSTM-Autoencoder for Anomaly Detection.** A two-layer LSTM-based autoencoder (Chapter 4) trained across distributed edge nodes using Federated Averaging (Chapter 5). The model learns normal system behavior from real-world telemetry (SMD dataset) and detects anomalies via reconstruction error, achieving an F1-score of 0.839 with an AUC-ROC of 0.950, significantly outperforming a centralized baseline (Section 11.2.2).

2. **Differential Privacy Integration.** Client-level gradient clipping and Gaussian noise injection provide formal DP guarantees for model updates shared during federated training. Experimental evaluation demonstrates the privacy–accuracy trade-off, with detection AUC-ROC decreasing from 0.972 (no DP) to 0.431 at the strictest tested privacy budget ($\varepsilon \approx 0.1$), establishing baseline behavior for DP-SGD in distributed telemetry environments (Section 11.2.3).

3. **Causal-Graph-Based Root Cause Analysis.** A causal dependency graph, constructed from distributed traces (Chapter 7), enables automated root cause identification when anomalies are detected. The PageRank-inspired scoring algorithm achieves a Top-1 accuracy of 42%, a Top-3 accuracy of 89%, and an MRR of 0.653, outperforming a random baseline (35% Top-1, 0.601 MRR) on the Train-Ticket microservice topology (Section 11.4.2).

4. **Q-Learning Adaptive Thresholding.** A per-service tabular Q-learning agent (Chapter 8) dynamically adjusts the anomaly detection threshold based on detection outcomes and SLO compliance feedback, reducing reliance on manually configured static thresholds. The agent's threshold trajectory demonstrates adaptation to sinusoidal traffic patterns over a 24-hour simulation, with ε-greedy exploration decaying from 0.10 to 0.05 (Section 11.5.2).

5. **Efficient FL Communication Stack.** An adaptive serialization and compression pipeline (Chapter 6) reduces model update payloads by up to 61% using Zstd compression on JSON/Protobuf payloads. For binary formats, the controller switches to LZ4 under high CPU load, providing 5.8× faster compression while sacrificing minimal payload reduction (Section 11.3.3).

6. **Fault-Tolerant Edge Design.** Two node profiles—lightweight (stateless) and standard (persistent SQLite cache)—enable crash recovery and seamless FL participation across heterogeneous edge deployments (Chapters 5 and 9).

12.2 Conclusion

The experimental evaluation (Chapter 11) demonstrates that the proposed framework achieves the following key results:

- **High accuracy on real-world data.** The federated LSTM-AE achieves an F1-score of 0.839 on the Server Machine Dataset, surpassing the centralized baseline (0.496). This confirms that the federated aggregation of distributed representations provides stronger generalization than single-node centralization on realistic telemetry. The evaluation also quantifies the DP trade-off, showing AUC-ROC gracefully degrading under strict privacy budgets.

- **Efficient communication.** Binary serialization (TorchSave) produces payloads 5.4× smaller than JSON or Protobuf. Zstd compression achieves an additional 9% reduction on binary payloads and up to 61% on text-based formats. The full FL communication pipeline—compression, transmission, and aggregation—accounts for less than 3% of total round time, with local training dominating at over 97%.

- **Actionable root cause identification.** The RCA engine identifies the true root cause in its top-3 ranking for 89% of fault-injection events on Train-Ticket, providing operators with a focused investigation set rather than an undifferentiated list of anomalous services. The framework consistently outperforms a random baseline in MRR (0.653 vs. 0.601).

- **Linear scalability.** FedAvg aggregation scales linearly from 5.1 ms at 5 nodes to 990.1 ms at 1 000 nodes, with bounded memory consumption ($\approx$ 42 MB). This empirical scalability confirms the coordinator is not a bottleneck up to at least 1 000 concurrent participants.

These results indicate that federated learning, differential privacy, and causal RCA can be combined into a practical, deployable system for anomaly detection in distributed microservice environments without requiring raw data centralization or manual threshold configuration.

12.3 Limitations

Despite the positive results, several limitations should be acknowledged:

- **Cold-start problem.** The Q-learning threshold agent requires an initial exploration phase during which alert quality may be suboptimal. The benchmark results show negative cumulative reward during the first 12 hours, indicating that a warm-up period is necessary before the policy stabilizes. Newly onboarded services experience this cold-start phase independently.

- **IID data assumption.** FedAvg assumes that local datasets are approximately IID across clients [58, 67]. In production deployments where edge nodes monitor services with substantially different workload characteristics, the aggregated global model may underperform compared to locally specialized models.

- **RCA accuracy on deep dependency chains.** The upstream analysis approach performs well when the root cause has few anomalous predecessors but struggles with services like `db-service` that sit at the bottom of many dependency chains. The Top-1 accuracy of 42% on Train-Ticket indicates room for more sophisticated causal inference methods.

- **DP noise and convergence.** Although detection AUC-ROC is acceptable under moderate DP, the training loss plateaus at $\approx$ 0.0014 compared to 0.0000 without DP, and extreme privacy budgets ($\varepsilon \approx 0.1$) degrade AUC-ROC to 0.431. For larger models or more complex anomaly patterns, this convergence gap translates into reduced detection sensitivity.

- **Single-coordinator architecture.** The current design relies on a single coordinator instance, which, despite crash-recovery mechanisms (Chapter 9), remains a potential availability bottleneck under extreme node counts.

12.4 Future Work

12.4.1 Transfer Learning

The current framework trains the LSTM-AE from scratch for each deployment. A promising direction is to pre-train the model on a large, generic corpus of system metric data—such as the Alibaba Cluster Trace dataset [104]—and then fine-tune on local edge data using few-shot adaptation. This would reduce the number of FL rounds required for convergence and improve detection quality for newly onboarded services that have limited local training data.

12.4.2 Hierarchical Federated Learning

For deployments exceeding 1 000 edge nodes, a two-tier hierarchical FL architecture would reduce the communication and computational load on the global coordinator. In this design:

- **Cluster-level sub-coordinators** aggregate model updates from nearby edge nodes within a geographic or administrative boundary.
- **The global coordinator** aggregates from sub-coordinators only, reducing the number of incoming connections by a factor proportional to the number of clusters.

This approach is compatible with the existing FedAvg aggregation logic and requires only routing changes rather than algorithmic modifications [51].

12.4.3 Enhanced Privacy Mechanisms

The current implementation uses central DP, where the coordinator is assumed to be honest-but-curious. Three complementary privacy mechanisms could strengthen the threat model:

- **Secure Multi-Party Computation (SMPC)**: Enables aggregation without revealing individual updates to the coordinator, removing the trusted-coordinator assumption [12].
- **Homomorphic Encryption**: Allows the coordinator to aggregate encrypted gradient updates without decryption, providing strong confidentiality guarantees at the cost of increased computational overhead.
- **Local Differential Privacy (LDP)**: Applies noise at the data collection level rather than at the gradient level, providing stronger per-sample privacy but typically requiring larger datasets for equivalent model utility [27].

12.4.4 Improved Root Cause Analysis

Several enhancements could improve RCA accuracy beyond the current Top-1 rate of 42%:

- **Multi-modal causal graph**: Incorporating structured log events and deployment metadata alongside trace data would provide richer causal signals. Error logs, memory allocation events, and container restarts could be modeled as additional edge types in the causal graph.
- **Temporal causal graph**: Annotating edges with propagation delays (derived from span durations) would enable the scoring algorithm to reason about temporal precedence, favoring services whose anomalies consistently precede downstream anomalies [33].
- **LLM-assisted incident analysis**: Large language models could be used to generate human-readable incident summaries from the RCA output, correlating root-cause rankings with relevant log snippets and suggesting remediation actions based on historical resolution patterns.

12.4.5 Deep Q-Network for Adaptive Thresholding

The current tabular Q-learning implementation discretizes the state space into coarse buckets, limiting the agent's ability to distinguish fine-grained detection regimes. Replacing the Q-table with a Deep Q-Network (DQN) [6] would enable:

- Continuous state representation without manual discretization.
- Richer state features (e.g., rolling reconstruction error statistics, request volume, time-of-day encoding).
- Potentially faster convergence through experience replay and target network stabilization.

This enhancement would be particularly valuable for services with complex, multi-modal traffic patterns where tabular discretization loses important signal.

12.4.6 Multi-Modal Anomaly Detection

The current LSTM-AE operates exclusively on numerical metric time series. Modern microservice deployments produce multiple telemetry modalities—metrics, structured logs, and distributed traces—each capturing different aspects of system behavior. Extending the LSTM-AE to jointly model these modalities through a multi-modal encoder (e.g., separate specialized encoders for each modality with

a shared latent space) could improve detection accuracy for complex anomalies that manifest across multiple telemetry types simultaneously. Recent datasets such as AnoMod [104] provide five-modality benchmarks suitable for evaluating such architectures.

References

[1] Martin Abadi et al. "Deep learning with differential privacy". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. 2016, pp. 308–318.

[2] Tanveer Ahmad, Zaid Al Ars, and H. Peter Hofstee. "Benchmarking Apache Arrow Flight – A wire-speed protocol for data transfer, querying and microservices". In: *arXiv preprint arXiv:2204.03032* (2022). URL: https://arxiv.org/abs/2204.03032.

[3] Alham Fikri Aji and Kenneth Heafield. "Sparse communication for distributed gradient descent". In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. ACL. 2017, pp. 440–445.

[4] Alibaba Group. *Alibaba Cluster Trace Program*. https://github.com/alibaba/clusterdata. 2022.

[5] Anonymous. "Adaptive thresholding for anomaly detection in dynamic environments". In: *ResearchGate article* (2024). Discusses adaptive threshold benefits and limitations.

[6] Anonymous. "XSQ-Learning: Adaptive similarity thresholds for accelerated and stable Q-Learning". In: *Applied Sciences* (2025). Covers Q-learning fundamentals and Bellman updates.

[7] Michèle Basseville and Igor V. Nikiforov. *Detection of Abrupt Changes: Theory and Application*. Prentice Hall, 1993.

[8] Mike Belshe, Roberto Peon, and Martin Thomson. *Hypertext Transfer Protocol Version 2 (HTTP/2)*. RFC 7540. Internet Engineering Task Force. 2015. DOI: 10.17487/RFC7540.

[9] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2 (1994), pp. 157–166.

[10] Betsy Beyer et al. *Site Reliability Engineering*. O'Reilly, 2016.

[11] D. Blyth et al. “ProIO: An Event-Based I/O Stream Format for Protobuf Messages”. In: *arXiv preprint arXiv:1812.03967* (2018). URL: https://arxiv.org/abs/1812.03967.

[12] Keith Bonawitz et al. “Towards federated learning at scale: System design”. In: *Proceedings of the 2nd SysML Conference*. 2019.

[13] Tim Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. 2017. DOI: 10.17487/RFC8259. URL: https://www.rfc-editor.org/rfc/rfc8259.

[14] Eric A Brewer. “Towards robust distributed systems”. In: *PODC*. Vol. 7. 2000, pp. 3434–3435.

[15] Sergey Brin and Lawrence Page. “The anatomy of a large-scale hypertextual Web search engine”. In: *Computer Networks and ISDN Systems*. Vol. 30. 1-7. Elsevier, 1998, pp. 107–117.

[16] Brendan Burns et al. “Borg, omega, and kubernetes”. In: *Queue* 14.1 (2016), pp. 70–93.

[17] Raghavendra Chalapathy and Sanjay Chawla. “Deep learning for anomaly detection: A survey”. In: *arXiv preprint arXiv:1901.03407* (2019).

[18] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly Detection: A Survey”. In: *ACM Computing Surveys* (2009).

[19] Yann Collet. *LZ4: Extremely Fast Compression Algorithm*. https://github.com/lz4/lz4. Project documentation and technical materials. 2011.

[20] Yann Collet. *LZ4: Extremely Fast Compression algorithm*. https://lz4.github.io/lz4/. 2026.

[21] Yann Collet and Murray Kucherawy. *Zstandard compression and the application/zstd media type*. RFC 8478. Internet Engineering Task Force (IETF), 2018.

[22] George Coulouris et al. *Distributed Systems: Concepts and Design*. 5th ed. Addison-Wesley, 2012.

[23] George F. Coulouris et al. *Distributed Systems: Concepts and Design*. 5th. Addison-Wesley, 2011.

[24] Rongxing Deng et al. “Optimal Workload Allocation in Fog-Cloud Computing”. In: *IEEE Internet of Things Journal* (2016).

[25] Nicola Dragoni et al. “Microservices: yesterday, today, and tomorrow”. In: *Present and Ulterior Software Engineering*. Springer, 2017, pp. 195–216.

[26] Cynthia Dwork. "Differential privacy". In: *International Colloquium on Automata, Languages, and Programming*. Springer. 2006, pp. 1–12.

[27] Cynthia Dwork and Aaron Roth. *Algorithmic Foundations of Differential Privacy*. Boston, MA, USA: Now Publishers Inc., 2014.

[28] Elastic NV. *ELK Stack: Elasticsearch, Logstash, Kibana*. https://www.elastic.co/what-is/elk-stack. 2013.

[29] Tolga Ergen and Suleyman S Kozat. "Unsupervised Anomaly Detection with LSTM Neural Networks". In: (2017).

[30] FlatBuffers Documentation. *FlatBuffers Documentation*. https://flatbuffers.dev/. Accessed: 2026-04-03. 2025.

[31] FlatBuffers Documentation. *FlatBuffers White Paper*. https://flatbuffers.dev/white_paper/. Accessed: 2026-04-03. 2025.

[32] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. "Model inversion attacks that exploit confidence information and basic countermeasures". In: *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 2015, pp. 1322–1333.

[33] Haoran Fu, Yuchen Wang, and Qi Li. "Root cause analysis in cloud microservice environments using causal graphs". In: *2020 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE. 2020, pp. 217–224.

[34] Felix A Gers and Jürgen Schmidhuber. "Learning to Forget: Continual Prediction with LSTM". In: (2000).

[35] Robin C Geyer, Tassilo Klein, and Moin Nabi. "Differentially private federated learning: A client-level perspective". In: *arXiv preprint arXiv:1712.07557* (2017).

[36] Seth Gilbert and Nancy Lynch. "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services". In: *ACM SIGACT News* 33.2 (2002), pp. 51–59.

[37] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[38] Ian Gorton. *Foundations of Scalable Systems: Designing Distributed Architectures*. 1st ed. O'Reilly Media, 2022.

[39] Grafana Labs. *Grafana: The Open Observability Platform*. https://grafana.com/. 2015.

[40] Alex Graves. "Supervised Sequence Labelling with Recurrent Neural Networks". In: (2012).

[41] Alex Graves and Jürgen Schmidhuber. “Framewise phoneme classification with bidirectional LSTM and other neural network architectures”. In: *Neural Networks* 18.5-6 (2005), pp. 602–610.

[42] Brendan Gregg. *BPF Performance Tools*. Addison-Wesley, 2019.

[43] gRPC Authors. *gRPC: A High Performance, Open Source Universal RPC Framework*. https://grpc.io/docs/. 2016.

[44] Prudhvi Gudla and Kamal Singh. “CBL: Compact Encoding of JSON-LD Data using CBOR and Bitmaps for Web of Things”. In: *arXiv preprint arXiv:2407.04398* (2024). URL: https://arxiv.org/abs/2407.04398.

[45] Geoffrey E Hinton and Ruslan R Salakhutdinov. “Reducing the dimensionality of data with neural networks”. In: *Science* 313.5786 (2006), pp. 504–507.

[46] Sepp Hochreiter and Jürgen Schmidhuber. “Long short-term memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

[47] Kyle Hundman et al. “Detecting Spacecraft Anomalies Using LSTMs”. In: *KDD* (2018).

[48] Jaeger Authors. *Jaeger Documentation*. https://www.jaegertracing.io/docs/. Accessed: 2026-04-03. 2024.

[49] Jaeger Authors. *Jaeger: Open Source Distributed Tracing*. https://www.jaegertracing.io/. 2017.

[50] Ian Jolliffe. *Principal Component Analysis*. 2016.

[51] Peter Kairouz, H. Brendan McMahan, Brendan Avent, et al. “Advances and Open Problems in Federated Learning”. In: *Foundations and Trends in Machine Learning* 14.1–2 (2021), pp. 1–210.

[52] Christos Kalloniatis et al. “Microservices Observability Challenges”. In: *Cloud Computing Research* (2020).

[53] Sai Praneeth Karimireddy et al. “Error Feedback Fixes SignSGD and other Gradient Compression Schemes”. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. 2019, pp. 3252–3261.

[54] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.

[55] Jakub Konečný et al. *Federated Learning: Strategies for Improving Communication Efficiency*. 2016. arXiv: 1610.05492 [cs.LG].

[56] Leslie Lamport. "Paxos Made Simple". In: *ACM SIGACT News* 32.4 (2001), pp. 18–25.

[57] Jun Li, Kai Zhu, and Hao Zhang. "MicroRank: Fault Localization in Microservices via Causal Dependency Graphs". In: *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2018, pp. 1–12.

[58] Tian Li et al. "Federated learning: Challenges, methods, and future directions". In: *IEEE Signal Processing Magazine* 37.3 (2020), pp. 50–60.

[59] Qingwei Lin. "AIOps: real-world challenges and research innovations". In: *2018 IEEE/ACM 40th International Conference on Software Engineering: Companion (ICSE-Companion)*. IEEE. 2018, pp. 4–5.

[60] Yujun Lin et al. "Deep gradient compression: Reducing the communication bandwidth for distributed training". In: *International Conference on Learning Representations*. 2018.

[61] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. "Isolation forest". In: *2008 Eighth IEEE International Conference on Data Mining*. IEEE. 2008, pp. 413–422.

[62] Wei Liu, Yang Chen, and et al. "Adaptive Anomaly Detection in Cloud Systems Using Reinforcement Learning". In: *IEEE International Conference on Cloud Computing (CLOUD)*. 2020, pp. 123–130.

[63] Naresh Lokiny. "Monitoring and Observability Solutions for Cloud-Native Applications". In: *European Journal of Applied Science Engineering and Technology* 10 (Feb. 2026), pp. 66–69. DOI: 10.5281/zenodo.13326379.

[64] Charity Majors, Liz Fong-Jones, and George Miranda. *Observability Engineering*. 2022.

[65] Pankaj Malhotra et al. "LSTM-based encoder-decoder for multi-sensor anomaly detection". In: *ICML 2015 Anomaly Detection Workshop*. Vol. 14. 2015, pp. 1–7.

[66] Yuyi Mao et al. "A Survey on Mobile Edge Computing: The Communication Perspective". In: *IEEE Communications Surveys & Tutorials* (2017).

[67] Brendan McMahan et al. "Communication-efficient learning of deep networks from decentralized data". In: *Artificial Intelligence and Statistics*. PMLR. 2017, pp. 1273–1282.

[68] Dirk Merkel. "Docker: lightweight linux containers for consistent development and deployment". In: *Linux journal* 2014.239 (2014), p. 2.

[69] Juan Muñoz and Alan Smith. “The impact of cascading failures in microservice architectures”. In: *Proceedings of the 10th ACM International Conference on Software Architecture*. ACM. 2017, pp. 45–54.

[70] Sam Newman. *Building Microservices*. 2nd ed. O’Reilly Media, 2021.

[71] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.

[72] Diego Ongaro and John Ousterhout. “In Search of an Understandable Consensus Algorithm”. In: *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*. 2014, pp. 305–319.

[73] OpenTelemetry Authors. *OpenTelemetry Documentation*. https://opentelemetry.io/docs/. Accessed: 2026-04-03. 2023.

[74] OpenZipkin Authors. *Zipkin Documentation*. https://zipkin.io/. Accessed: 2026-04-03. 2024.

[75] OpenZipkin Project. *Zipkin: Distributed Tracing System*. https://zipkin.io/. 2012.

[76] Lawrence Page et al. *The PageRank citation ranking: Bringing order to the web.* Tech. rep. Stanford InfoLab, 1999.

[77] Guansong Pang et al. “Deep learning for anomaly detection: A review”. In: *ACM Computing Surveys* 54.2 (2021), pp. 1–38.

[78] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council*. General Data Protection Regulation (GDPR). 2016.

[79] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. “On the difficulty of training recurrent neural networks”. In: *International Conference on Machine Learning*. PMLR. 2013, pp. 1310–1318.

[80] Jon Postel. *Transmission Control Protocol*. RFC 793. Internet Engineering Task Force. Sept. 1981. DOI: 10.17487/RFC0793.

[81] Jon Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force. 1980. DOI: 10.17487/RFC0768.

[82] Prometheus Authors. *Prometheus: Monitoring System and Time Series Database*. https://prometheus.io/. 2016.

[83] Protocol Buffers Documentation. *Protocol Buffers Overview*. https://protobuf.dev/overview/. Accessed: 2026-04-03. 2025.

[84] Michel Raynal. *Fault-Tolerant Message-Passing Distributed Systems: An Algorithmic Approach*. Springer, 2018.

[85] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536.

[86] Mahadev Satyanarayanan. "The Emergence of Edge Computing". In: *Computer* 50.1 (2017), pp. 30–39.

[87] Bernhard Schölkopf et al. "Estimating the support of a high-dimensional distribution". In: *Neural Computation*. Vol. 13. 7. 2001, pp. 1443–1471.

[88] Mike Schuster and Kuldip Paliwal. "Bidirectional Recurrent Neural Networks". In: (1997).

[89] Weisong Shi et al. "Edge computing: Vision and challenges". In: *IEEE Internet of Things Journal* 3.5 (2016), pp. 637–646.

[90] Reza Shokri et al. "Membership inference attacks against machine learning models". In: *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE. 2017, pp. 3–18.

[91] Benjamin H Sigelman et al. *Dapper, a large-scale distributed systems tracing infrastructure*. Tech. rep. Google, 2010.

[92] Sebastian U. Stich, Jean-Baptiste Cordonnier, and Martin Jaggi. "Sparsified SGD with Memory". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2018.

[93] Ya Su et al. "Robust anomaly detection for multivariate time series through stochastic recurrent neural networks". In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, pp. 2828–2837.

[94] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd ed. MIT Press, 2018.

[95] Andrew S. Tanenbaum and Maarten van Steen. *Distributed Systems: Principles and Paradigms*. 2nd ed. Pearson Prentice Hall, 2007.

[96] Martin Thomson, Mark Benfield, and Roberto Peon. *HTTP/2*. RFC 9113. Internet Engineering Task Force. June 2022. DOI: 10.17487/RFC9113.

[97] James Turnbull. *The Prometheus Monitoring System*. 2018.

[98] Bimal Varghese and Rajkumar Buyya. “Challenges and opportunities in edge computing”. In: *2016 IEEE International Conference on Smart Cloud* (2016), pp. 20–26.

[99] Juan Cruz Viotti and Mital Kinderkhedia. “A Benchmark of JSON-compatible Binary Serialization Specifications”. In: *arXiv preprint arXiv:2201.03051* (2022). DOI: 10.48550/arXiv.2201.03051.

[100] Christopher JCH Watkins and Peter Dayan. “Q-learning”. In: *Machine learning* 8.3-4 (1992), pp. 279–292.

[101] Jia Xu et al. “Federated learning for healthcare informatics”. In: *Journal of Healthcare Informatics Research* 5 (2021), pp. 1–27.

[102] Xue Yang, Enda Howley, and Michael Schukat. “Agent-based dynamic thresholding for adaptive anomaly detection using reinforcement learning”. In: *Neural Computing and Applications* (2024). DOI: 10.1007/s00521-024-10536-0.

[103] Chen Yu et al. “Distributed Learning over Unreliable Networks”. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research. 2019, pp. 7202–7212.

[104] Li Zhang, Ming Chen, and Wei Liu. “Tracing microservices for root cause analysis: A survey”. In: *Journal of Systems and Software* 155 (2019), pp. 92–108.

[105] Liang Zhang, Yong Zhang, and et al. “Root Cause Analysis for Microservices Using Distributed Traces”. In: *IEEE Transactions on Services Computing*. Vol. 12. 6. 2019, pp. 931–945.

[106] Chong Zhou and Randy Paffenroth. “Anomaly Detection with Robust Deep Autoencoders”. In: (2017).

[107] Xiang Zhou et al. “Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study”. In: *IEEE Transactions on Software Engineering* 47.2 (2018), pp. 243–260.